

The topology of software risk in scientific models

3. Application to domain models

Arnald Puy

Contents

1	Preliminary	2
2	Uncertainty and sensitivity analysis	9
3	Analysis of git logs	44
4	Session information	49

1 Preliminary

```
# PRELIMINARY FUNCTIONS #####
#####

sensobol::load_packages(c("data.table", "tidyverse", "openxlsx", "scales",
                          "cowplot", "readxl", "ggrepel", "tidytext", "here",
                          "tidygraph", "igraph", "foreach", "parallel", "ggraph",
                          "tools", "purrr", "sensobol", "benchmarkme", "softwareRisk"))

# Select color palette -----

color_languages <- c("fortran" = "steelblue", "python" = "lightgreen")

# Color to functional forms -----

color_functional_forms <- c("additive" = "#C65D09", "compensatory" = "#3B5B92")

# Set seed -----

seed <- 123

# Source all .R files in the "functions" folder -----

r_functions <- list.files(path = here("functions"), pattern = "\\R$", full.names = TRUE)
lapply(r_functions, source)

# CREATE DATASET #####

# Path to folder -----

path <- "./datasets/call_metrics"

# List CSV files -----

files <- list.files(path, pattern = "\\csv$", full.names = TRUE)

# Split by language -----

python_files <- grep("python", files, value = TRUE, ignore.case = TRUE)
fortran_files <- grep("fortran", files, value = TRUE, ignore.case = TRUE)

base_fortran <- file_path_sans_ext(basename(fortran_files))
base_python <- file_path_sans_ext(basename(python_files))

model_names_fortran <- models <- sub(".*_", "", base_fortran)
model_names_python <- models <- sub(".*_", "", base_python)
```

```

# Load and name files -----

python_list <- lapply(python_files, fread)
fortran_list <- lapply(fortran_files, fread)

names(python_list) <- model_names_python
names(fortran_list) <- model_names_fortran

# RBIND -----

make_callgraph <- function(lst, lang) {
  rbindlist(lst, idcol = "model") %>%
    .[, language := lang] %>%
    .[, .(file, model, language, `function`, call)] %>%
    setnames(., c("function", "call"), c("from", "to"))
}

python_callgraphs <- make_callgraph(python_list, "python")
fortran_callgraphs <- make_callgraph(fortran_list, "fortran")

all_callgraphs <- rbind(python_callgraphs, fortran_callgraphs)

# SOURCE CODE CLASSIFICATION BY FUNCTIONAL ROLE #####

# Strip leading "./models/"-----
all_callgraphs[, file_clean:= sub("^\\.models/", "", file)]

# Provenance: file must be inside ./models to be eligible at all-----
all_callgraphs[, in_model_tree:= !is.na(file) & nchar(file) > 0L &
  grepl("^\\.models/", file)]

# Rest = everything after first segment-----
all_callgraphs[, rest:= sub("^[/]+/", "", file_clean)]

# If rest starts with model_id/ then drop it (this is the duplicate)-----
all_callgraphs[, rest:= fifelse(startsWith(rest, paste0(tolower(model), "/")),
  sub("^[/]+/", "", rest), rest)]

# In case of triple nesting like vic/vic/vic/...-----
all_callgraphs[, rest:= fifelse(startsWith(rest, paste0(tolower(model), "/")),
  sub("^[/]+/", "", rest), rest)]

# Top_level-----
all_callgraphs[, top_level:= tstrsplit(rest, "/", fixed = TRUE, keep = 1L)]

# Useful generic external patterns (works across models)-----
all_callgraphs[, is_generic_external:= in_model_tree & (

```

```

grepl("(^|/)(extern|external|externals|third[_-]?party|vendor|vendored)(/|$)",
      rest, ignore.case = TRUE) |
grepl("(^|/)\\.lib(/|$)", rest, ignore.case = TRUE))]

# CTSM-specific external (framework + FoX parser under cdeps + CESM shared infra)
all_callgraphs[, is_ctsm_external:= (model == "CTSM") & in_model_tree & (
  grepl("^cime(/|$)", rest) |
  grepl("^cime_config(/|$)", rest) |
  grepl("^components/cdeps(/|$)", rest) |      # FoX XML/SAX parser
  grepl("(^|/)share_esmf(/|$)", rest) |        # shared ESMF infrastructure
  grepl("^src/unit_test_shr(/|$)", rest)       # unit tests
)]

# Extra CTSM allowlist (tightens "model core" to land-model code)-----
CTSM_STRICT <- TRUE

all_callgraphs[, is_ctsm_allowed:= TRUE]

if (CTSM_STRICT) {
  all_callgraphs[model == "CTSM" & in_model_tree, is_ctsm_allowed :=
    grepl("^src(/|$)", rest) |
    grepl("^lilac(/|$)", rest) |
    grepl("^components/cism(/|$)", rest)]
}

## Component classification (order matters)-----
all_callgraphs[nchar(file) == 0L | is.na(file), component:= NA_character_]

all_callgraphs[!is.na(file) & nchar(file) > 0L, component:= fcase(

  # Anything not in ./models is external immediately
  !in_model_tree, "external_lib",

  # Generic external dirs (vendored/third_party/etc.)
  is_generic_external, "external_lib",

  # CI and vendored libraries
  top_level == ".github", "ci_cd",
  top_level == ".lib" | grepl("/\\.lib/", rest), "vendored_lib",

  # CIME / CESM / CDEPS infrastructure (framework)
  grepl("^cime/CIME/", rest), "framework",
  grepl("^cime_config/", rest), "framework",
  grepl("^cime/doc/", rest), "framework",
  grepl("^components/cdeps/cime_config/", rest), "framework",

  # CTSM-specific: treat cime + cdeps (incl FoX) as external/framework

```

```

is_ctsm_external, "framework",

# CTSM shared "shr_*" routines (framework by symbol)
(model == "CTSM") & (grepl("^shr_", from) | grepl("^shr_", to)), "framework",

# Tests (incl. SystemTests, case-insensitive)
grepl("SystemTests/", rest, ignore.case = TRUE), "tests",
grepl("/tests?/|^tests?/", rest, ignore.case = TRUE), "tests",
grepl("(^|/)unit_test", rest, ignore.case = TRUE), "tests",

# Drivers: Fortran code clearly in drivers directories
grepl("(^|/)drivers(/|$)", rest) & language == "fortran", "driver",

# Couplers: NUOPC / LILAC / CESM / cpl
grepl("cpl_|/cesm|/cpl|/cpl_", rest), "coupler",

# CLI / tools: scripts, setup, CTSM python utilities
grepl("tools/|scripts/|cli\\.py$|setup\\.py$", rest), "cli_or_tool",
grepl("^python/ctsm/", rest), "cli_or_tool",

# Everything else: model core
default = "model_core"
)]

# include flag for risk calculations -----
# Key changes:
# - require in_model_tree
# - exclude external_lib/framework/tests/etc (already via component filter)
# - CTSM strict allowlist (optional)
all_callgraphs[, include_in_risk := (
  in_model_tree &
  language %chin% c("fortran", "python") &
  component %chin% c("model_core", "coupler") &
  (model != "CTSM" | !CTSM_STRICT | is_ctsm_allowed)
)]

# --- Sanity check: should now be FALSE for the SAX/FoX functions in CTSM
all_callgraphs[
  model == "CTSM" & include_in_risk &
  (grepl("sax|parseDTD|parsefile|parsestring|runParser", from, ignore.case = TRUE) |
   grepl("sax|parseDTD|parsefile|parsestring|runParser", to, ignore.case = TRUE)),
  .(from, to, file, rest, component)
]

## Empty data.table (0 rows and 5 cols): from,to,file,rest,component
# Summarize -----

```

```
all_callgraphs[, .N, component]
```

```
##      component      N
##      <char> <int>
## 1:      ci_cd       5
## 2: external_lib    100
## 3:   framework  8603
## 4:      tests    194
## 5: cli_or_tool    820
## 6:  model_core 15727
## 7:      driver     64
## 8:      coupler   299
```

```
all_callgraphs[, .N, include_in_risk]
```

```
##   include_in_risk      N
##           <lgcl> <int>
## 1:           FALSE  9786
## 2:           TRUE 16026
```

```
# Remove module calls -----
```

```
all_callgraphs <- all_callgraphs[!(from %in% "<module>")] %>%
  .[include_in_risk == TRUE]
```

```
# LOAD CYCLOMATIC COMPLEXITY VALUES FOR FUNCTIONS AND SUBROUTINES #####
```

```
cc_unique <- fread("./datasets/cyclomatic_complexity_functions.csv")
```

```
# CREATE NETWORK FROM CALL GRAPHS #####
```

```
all_graphs <- all_callgraphs[, .(graph = list(as_tbl_graph(.SD, directed = TRUE))),
  model]
```

```
# ADD NODE METRICS #####
```

```
# Define the weights to characterize risky nodes -----
```

```
alpha <- 0.6 # Weight to cyclomatic complexity
beta  <- 0.3 # Weight to in-degree (impact of bug upstream)
gamma <- 0.1 # Weight to betweenness (critical bridge)
```

```
# Add node metrics -----
```

```
all_graphs[, graph:= Map(function(g, m) {
```

```
  comp_sub <- cc_unique[model == m]
```

```
  # mean cyclomatic complexity for this model & language -----
```

```

mean_cyclo <- mean(comp_sub$cyclomatic_complexity, na.rm = TRUE)

g %>%
  activate(nodes) %>%

  # Left join with dataset with cyclomatic complexity values -----
  left_join(comp_sub, by = "name") %>%

  # replace NA cyclomatic_complexity with model-language mean -----
  mutate(cyclomatic_complexity = if (!is.na(mean_cyclo)) {

    ifelse(is.na(cyclomatic_complexity), mean_cyclo, cyclomatic_complexity)

  } else {

    # if even the mean is NA (all NA in comp_sub), leave as-is

    cyclomatic_complexity
  }) %>%

  # Remove Python MODULE_AGG / CLASS_AGG nodes from this graph
  # because they are not callable -----

  filter(!(language == "python" & type %in% c("MODULE_AGG", "CLASS_AGG"))) %>%

  # Calculation of key network metrics -----

  mutate(type = type,
    indeg = centrality_degree(mode = "in"),
    outdeg = centrality_degree(mode = "out"),
    btw = centrality_betweenness(directed = TRUE, weights = NULL),
    cyclo_sc = rescale(cyclomatic_complexity),
    indeg_sc = rescale(indeg),
    btw_sc = rescale(btw),
    risk_score = alpha * cyclo_sc * beta * indeg_sc * gamma * btw_sc)
},
graph, model)]

# EXTRACT NODE DF #####

all_graphs[, node_df := lapply(graph, as_tibble, what = "nodes")]

# Export full node df -----

```

```

full_node_df <- all_graphs %>%
  mutate(node_df = purrr::map(node_df, ~ select(.x, -model, -language))) %>%
  unnest(node_df) %>%
  select(-graph) %>%
  data.table()

write.xlsx(full_node_df, "full_node_df.xlsx")

# COMPUTE ALL PATHS AND THEIR RISK SCORES #####

# list of risk-form specifications -----
# We run an additive and a power-mean version with p = -1

risk_specs <- list(additive = list(risk_form = "additive", p = NA),
  power_mean = list(risk_form = "power_mean", p = -1))

# Run programmatically -----

for (nm in names(risk_specs)) {
  spec <- risk_specs[[nm]]

  all_graphs[, (paste0("paths_tbl_", nm)) := Map(all_paths_fun, graph = graph,
    risk_form = spec$risk_form,
    alpha = alpha,
    beta = beta,
    gamma = gamma,
    p = spec$p,
    complexity_col = "cyclomatic_complexity")]]}

# Unnest tables -----

# For additive form -----

nodes_additive <- unnest_paths_tbl_fun(all_graphs, "paths_tbl_additive", "nodes") %>%
  .[, risk_form := "additive"]
paths_additive <- unnest_paths_tbl_fun(all_graphs, "paths_tbl_additive", "paths") %>%
  .[, risk_form := "additive"]

# For power mean form -----

nodes_power_mean <- unnest_paths_tbl_fun(all_graphs, "paths_tbl_power_mean", "nodes") %>%
  .[, risk_form := "power_mean"]
paths_power_mean <- unnest_paths_tbl_fun(all_graphs, "paths_tbl_power_mean", "paths") %>%
  .[, risk_form := "power_mean"]

# Rbind -----

```



```

nodes_all <- rbind(nodes_additive, nodes_power_mean, fill = TRUE)
paths_all <- rbind(paths_additive, paths_power_mean, fill = TRUE)[, path_str:= NULL]

# Export -----

write.xlsx(nodes_all, "full_nodes_df.xlsx")
write.xlsx(paths_all, "full_paths_df.xlsx")

```

2 Uncertainty and sensitivity analysis

```

# CONDUCT UNCERTAINTY AND SENSITIVITY ANALYSIS #####

# Define sample size and order of effects -----

N <- 2^10
order <- "first"

# Run the function (we remove the vic and python model implementation because
# there are not paths) -----

idx <- all_graphs[model != "VIC", which = TRUE]

# Define variants (input column -> output column + extra args) -----

ua_specs <- list(additive = list(paths_col = "paths_tbl_additive",
                                out_col = "uncertainty_sensitivity_additive",
                                extra = list()),
                power_mean = list(paths_col = "paths_tbl_power_mean",
                                out_col = "uncertainty_sensitivity_power_mean",
                                extra = list(risk_form = "power_mean")))

# Define parallel settings -----

ncores <- max(1L, parallel::detectCores() - 1L)

# Run UA/SA -----

for (nm in names(ua_specs)) {
  spec <- ua_specs[[nm]]

  all_graphs[idx, (spec$out_col):= parallel::mclapply(X = get(spec$paths_col),
                                                    FUN = function(x)
                                                    do.call(uncertainty_fun,
                                                            c(list(all_paths_out = x,
                                                                N = N,
                                                                order = order),

```

```

                                spec$extra)),
                                mc.cores = ncores
  )]
}

# Extract data -----

# Additive -----

nodes_us_additive <- unnest_us_fun(all_graphs,
                                us_col = "uncertainty_sensitivity_additive",
                                slot    = "nodes") %>%

  .[, risk_form:= "additive"]

paths_us_additive <- unnest_us_fun(all_graphs,
                                us_col = "uncertainty_sensitivity_additive",
                                slot    = "paths") %>%

  .[, risk_form:= "additive"]

# Power-mean -----

nodes_us_power_mean <- unnest_us_fun(all_graphs,
                                us_col = "uncertainty_sensitivity_power_mean",
                                slot    = "nodes") %>%

  .[, risk_form:= "power_mean"]

paths_us_power_mean <- unnest_us_fun(all_graphs,
                                us_col = "uncertainty_sensitivity_power_mean",
                                slot    = "paths") %>%

  .[, risk_form:= "power_mean"]

# Rbind -----

nodes_us_all <- rbind(nodes_us_additive, nodes_us_power_mean, fill = TRUE)
paths_us_all <- rbind(paths_us_additive, paths_us_power_mean, fill = TRUE)

# Calculate median, q5 and q95 -----

cols <- c("uncertainty_analysis", "risk_trend", "gini_index")

for (nm in cols) {

  prefix <- switch(nm, uncertainty_analysis = "P", risk_trend = "risk",
                  gini_index = "gini")
  paths_us_all[, paste0(prefix, "_median"):= vapply(get(nm), median, numeric(1), na.rm = TRUE)]
  paths_us_all[, paste0(prefix, "_q5"):= vapply(get(nm), quantile, numeric(1), probs = 0.05, na.rm = TRUE)]
  paths_us_all[, paste0(prefix, "_q95"):= vapply(get(nm), quantile, numeric(1), probs = 0.95, na.rm = TRUE)]
}

```

```

}

# Extract sensitivity indices -----

indices_all <- extract_sa_fun(nodes_us_all)

# CALCULATE SOME DESCRIPTIVE METRICS #####

tmp <- data.table(paths_all[risk_form == "additive"])[, .(n_paths = .N), model] %>%
  .[order(-n_paths)]

tmp2 <- data.table(nodes_all[risk_form == "additive"])[, .(n_nodes = .N), model] %>%
  .[order(-n_nodes)]

# Path to node ratio: how interconnected the model is.
# Model_cc: Proxy for algorithmic complexity of model.
# Avg_path_length: Proxy for depth of dependency chains (risk-highway potential)
# Model fragility: more (error) propagation routes.
models_metrics <- merge(tmp, tmp2) %>%
  .[, `:=`(path_to_node_ratio = n_paths / n_nodes,
           model_cc = n_paths / log(n_nodes),
           avg_path_length = n_nodes / log(n_paths + 1),
           model_fragility_index = n_paths / (n_nodes * (n_nodes - 1)))]

models_metrics

# Read descriptive_stats_file -----

num_cols <- c("files", "functions", "modules", "lines", "lines_code", "lines_comments")

descriptive_stats <- data.table(read_xlsx("./datasets/descriptive_statistics/descriptive_statistics.xlsx"))

descriptive_stats <- dcast(melt(descriptive_stats, id.vars="model", measure.vars=num_cols),
                          model ~ variable, value.var = "value", fun.aggregate = function(z) sum(z))
  .[, lines_function := lines_code / functions]

all_descriptive_df <- merge(models_metrics, descriptive_stats)

# Sort by model -----

model_ordered <- all_descriptive_df[, sum(lines), model] %>%
  .[order(V1)]

# Plot descriptive measures per model -----

plot_descriptive <- melt(all_descriptive_df, measure.vars = c("lines_code", "n_nodes", "n_paths",
                                                             "path_to_node_ratio", "model_cc"))

```

```

[, model:= factor(model, levels = model_ordered[, model])] %>%
ggplot(., aes(model, value)) +
geom_col() +
coord_flip() +
scale_y_continuous(breaks = breaks_pretty(n = 2)) +
scale_fill_manual(values = color_languages, name = "") +
facet_wrap(~ variable, ncol = 7, scales = "free_x") +
labs(x = "", y = "N") +
theme_AP() +
theme(legend.position = c(0.1, 0.3))

```

```

## Warning in melt.data.table(all_descriptive_df, measure.vars = c("lines_code", :
## 'measure.vars' [lines_code, n_nodes, n_paths, path_to_node_ratio, ...] are not
## all of the same type. By order of hierarchy, the molten data value column will
## be of type 'double'. All measure variables not of type 'double' will be coerced
## too. Check DETAILS in ?melt.data.table for more on coercion.

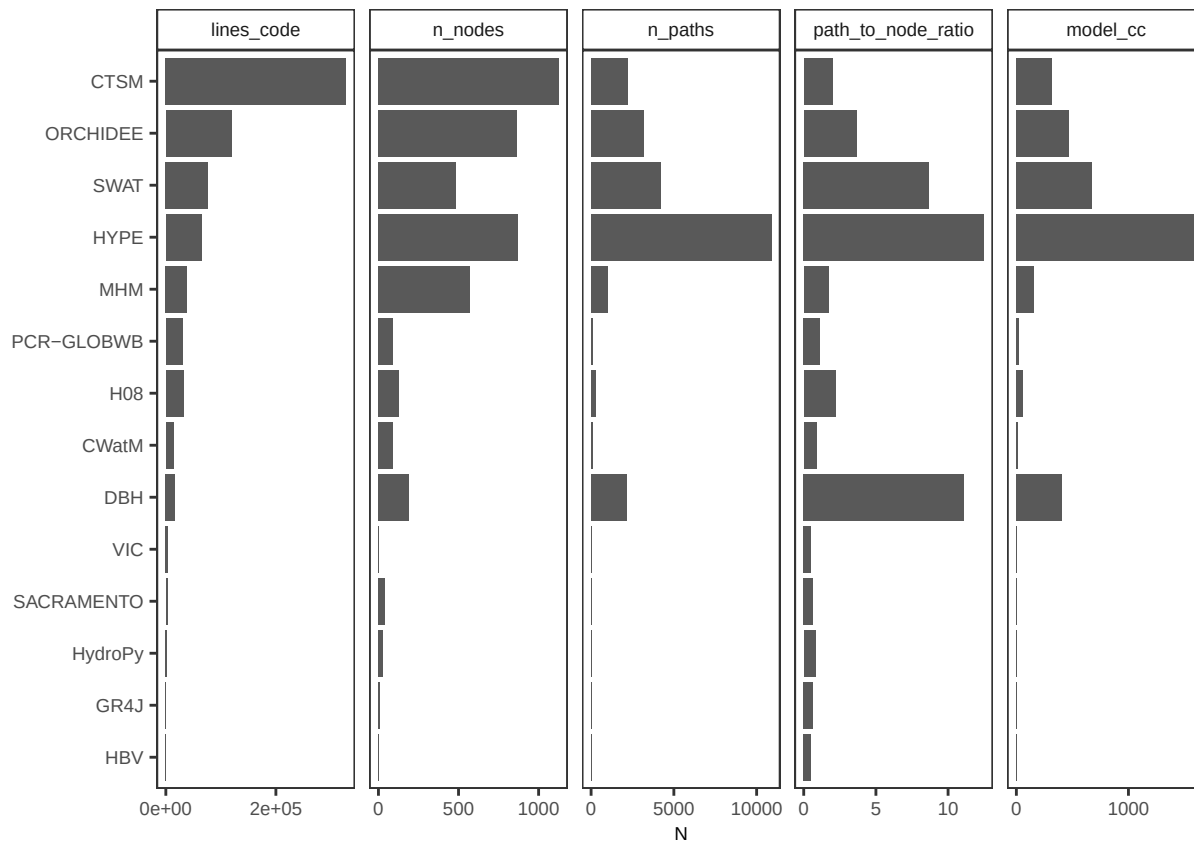
```

```
plot_descriptive
```

```

## Warning: No shared levels found between `names(values)` of the manual scale and the
## data's fill values.

```



```
# METRICS AT THE FILE AND FUNCTION LEVEL #####
```

```
folder <- "./datasets/results_per_function"
```

```

# Get names of files -----
csv_files <- list.files(path = folder, pattern = "\\*.csv$", full.names = TRUE)

# Split into file_metrics and func_metrics -----

file_metric_files <- grep("file_metrics", csv_files, value = TRUE)
func_metric_files <- grep("func_metrics", csv_files, value = TRUE)

# Build one named list -----

list_metrics <- list(file_metrics = setNames(lapply(file_metric_files, fread),
                                              basename(file_metric_files)),
                    func_metrics = setNames(lapply(func_metric_files, fread),
                                              basename(func_metric_files)))

# Create function to combine files -----

make_combined <- function(subset_list, pattern) {
  rbindlist(subset_list[grep(pattern, names(subset_list))], idcol = "source_file")
}

# Combine files -----

metrics_combined <- list(file_fortran = make_combined(list_metrics$file_metrics, "fortran"),
                        file_python = make_combined(list_metrics$file_metrics, "python"),
                        func_fortran = make_combined(list_metrics$func_metrics, "fortran"),
                        func_python = make_combined(list_metrics$func_metrics, "python"))

# Functions to extract name of model and language from file -----

extract_model <- function(x)
  sub("^^(file|func)_metrics_\\d+_[A-Za-z0-9-]+_(fortran|python).*", "\\2", x)

extract_lang <- function(x)
  sub("^^(file|func)_metrics_\\d+_[A-Za-z0-9-]+_(fortran|python).*", "\\3", x)

# Extract name of model and language -----

metrics_combined <- lapply(metrics_combined, function(dt) {
  dt[, source_file := sub("\\*.csv$", "", basename(source_file))]
  dt[, model := extract_model(source_file)]
  dt[, language := extract_lang(source_file)]
  dt
})

```

```

# Add column of complexity category -----

metrics_combined <- lapply(names(metrics_combined), function(nm) {
  dt <- as.data.table(metrics_combined[[nm]])
  if (grepl("^func_", nm) && "cyclomatic_complexity" %in% names(dt)) {
    dt[, complexity_category := cut(
      cyclomatic_complexity,
      breaks = c(-Inf, 10, 20, 50, Inf),
      labels = c("b1", "b2", "b3", "b4")
    )]
  }
  dt
}) |> setNames(names(metrics_combined))

# Define labels -----

lab_expr <- c(b1 = expression(C %in% "(" * 0 * ", 10" * "]),
  b2 = expression(C %in% "(" * 10 * ", 20" * "]),
  b3 = expression(C %in% "(" * 20 * ", 50" * "]),
  b4 = expression(C %in% "(" * 50 * ", " * infinity * ")"))

# Define vector to exclude classes that are not functions -----

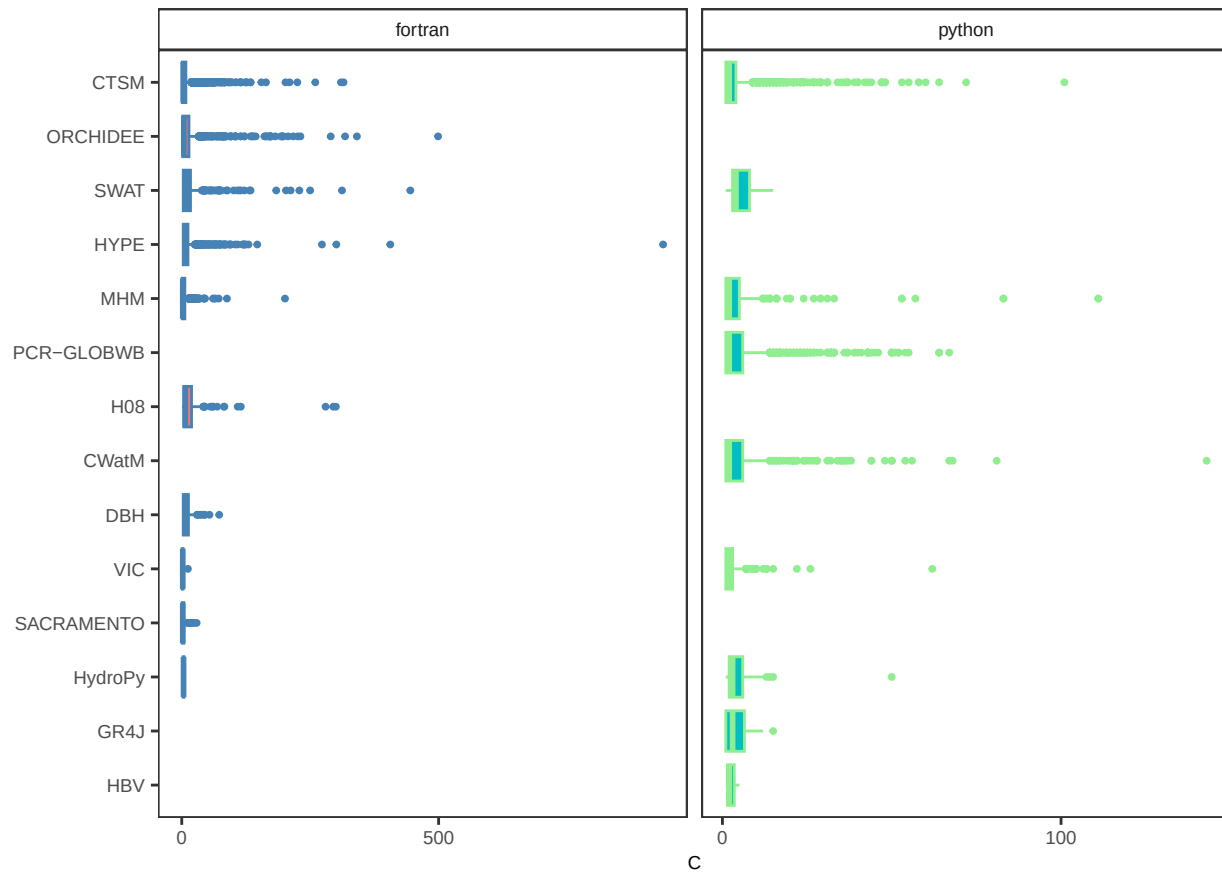
excluded_classes_vec <- c("MODULE_AGG", "CLASS_AGG")

# PLOT #####

plot_c_model <- metrics_combined[grepl("^func_", names(metrics_combined))] %>%
  lapply(., function(x)
    x[, .(model, language, `function`, cyclomatic_complexity, loc, bugs, type)] %>%
    rbindlist() %>%
    .[!type %in% excluded_classes_vec] %>%
    .[, model := factor(model, levels = model_ordered[, model])] %>%
    na.omit() %>%
    ggplot(., aes(model, cyclomatic_complexity, fill = language, color = language)) +
    geom_boxplot(outlier.size = 0.7) +
    coord_flip() +
    scale_y_continuous(breaks = scales::breaks_pretty(n = 2)) +
    facet_wrap(~language, scales = "free_x") +
    labs(x = "", y = "C") +
    theme_AP() +
    scale_color_manual(values = color_languages) +
    theme(legend.position = "none",
      plot.margin = margin(0, 2, 0, 0))

plot_c_model

```



```
## ----plot_scatter_and_bar, dependson="read_metrics_function_data", fig.height=2.5, fig.width=10
```

```
# Scatterplot cyclomatic vs lines of code -----
```

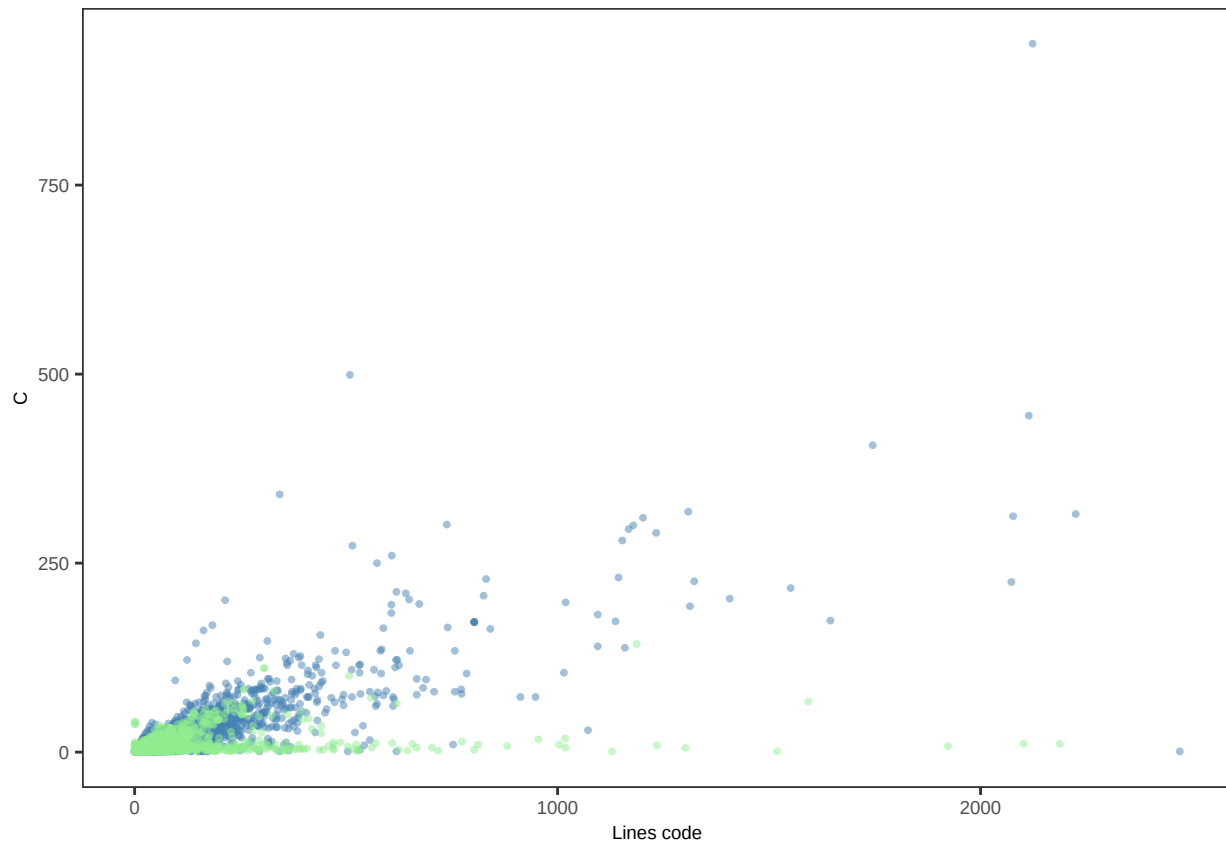
```
plot_c_vs_loc <- metrics_combined[grep("^func_", names(metrics_combined))] %>%
  lapply(., function(x) x[, .(loc, cyclomatic_complexity, language)]) %>%
  rbindlist() %>%
  ggplot(., aes(loc, cyclomatic_complexity, color = language)) +
  geom_point(alpha = 0.5, size = 0.7) +
  scale_x_continuous(breaks = breaks_pretty(n = 3)) +
  labs(x = "Lines code", y = "C") +
  scale_color_manual(values = color_languages) +
  theme_AP() +
  scale_x_continuous(breaks = breaks_pretty(n = 2)) +
  theme(legend.position = "none")
```

```
## Scale for x is already present.
```

```
## Adding another scale for x, which will replace the existing scale.
```

```
plot_c_vs_loc
```

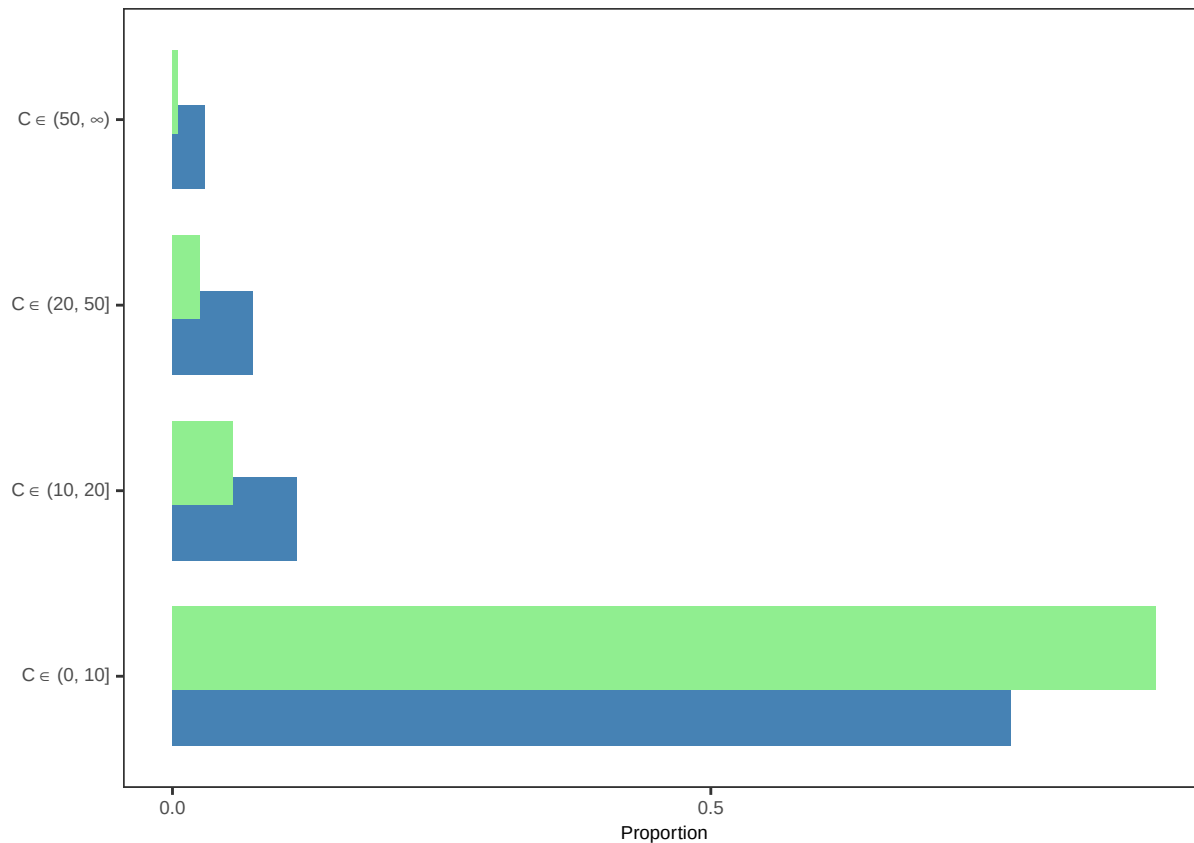
```
## Warning: Removed 1195 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



```
# Count & proportion -----

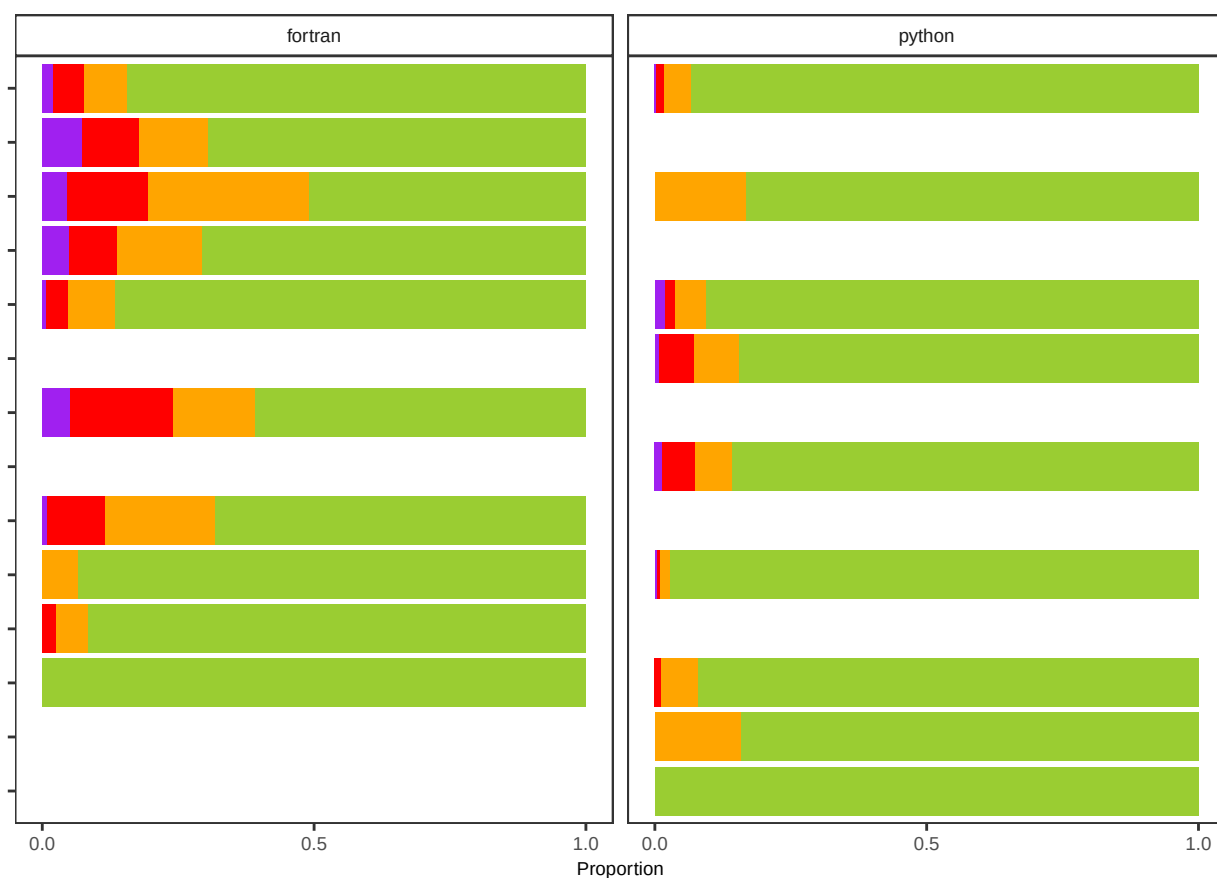
plot_bar_cyclomatic <- metrics_combined[grep("^func_", names(metrics_combined))] %>%
  lapply(., function(x) x[, .(complexity_category, language, type)]) %>%
  rbindlist() %>%
  .[!type %in% excluded_classes_vec] %>%
  .[, .N, .(complexity_category, language)] %>%
  .[, proportion := N / sum(N), language] %>%
  ggplot(., aes(complexity_category, proportion, fill = language)) +
  geom_bar(stat = "identity", position = position_dodge(0.6)) +
  scale_fill_manual(values = color_languages) +
  scale_y_continuous(breaks = scales::breaks_pretty(n = 3)) +
  scale_x_discrete(labels = lab_expr) +
  labs(x = "", y = "Proportion") +
  coord_flip() +
  theme_AP() +
  theme(legend.position = "none")

plot_bar_cyclomatic
```

```
plot_bar_category <- metrics_combined[grep("^func_", names(metrics_combined))] %>%
  lapply(., function(x)
    x[, .(model, language, complexity_category, type)] %>%
    rbindlist() %>%
    .[!type %in% excluded_classes_vec] %>%
    .[, model := factor(model, levels = model_ordered[, model])] %>%
    .[, .N, .(model, language, complexity_category)] %>%
    .[, proportion := N / sum(N), .(language, model)] %>%
    ggplot(., aes(model, proportion, fill = complexity_category)) +
    geom_bar(stat = "identity") +
    scale_fill_manual(values = c("yellowgreen", "orange", "red", "purple"),
                      labels = lab_expr,
                      name = "") +
    facet_wrap(~language) +
    labs(x = "", y = "Proportion") +
    coord_flip() +
    scale_y_continuous(breaks = scales::breaks_pretty(n = 3)) +
    theme_AP() +
    theme(legend.position = "none") +
    theme(axis.text.y = element_blank(),
          legend.text = element_text(size = 7),
          plot.margin = margin(0, 0, 0, 2))
```

plot_bar_category



```
# MERGE FIGURES #####
```

```
legend2 <- get_legend_fun(plot_bar_category + theme(legend.position = "top"))
```

```
## Warning: `is.ggplot()` was deprecated in ggplot2 3.5.2.
```

```
## i Please use `is_ggplot()` instead.
```

```
## This warning is displayed once per session.
```

```
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
```

```
## generated.
```

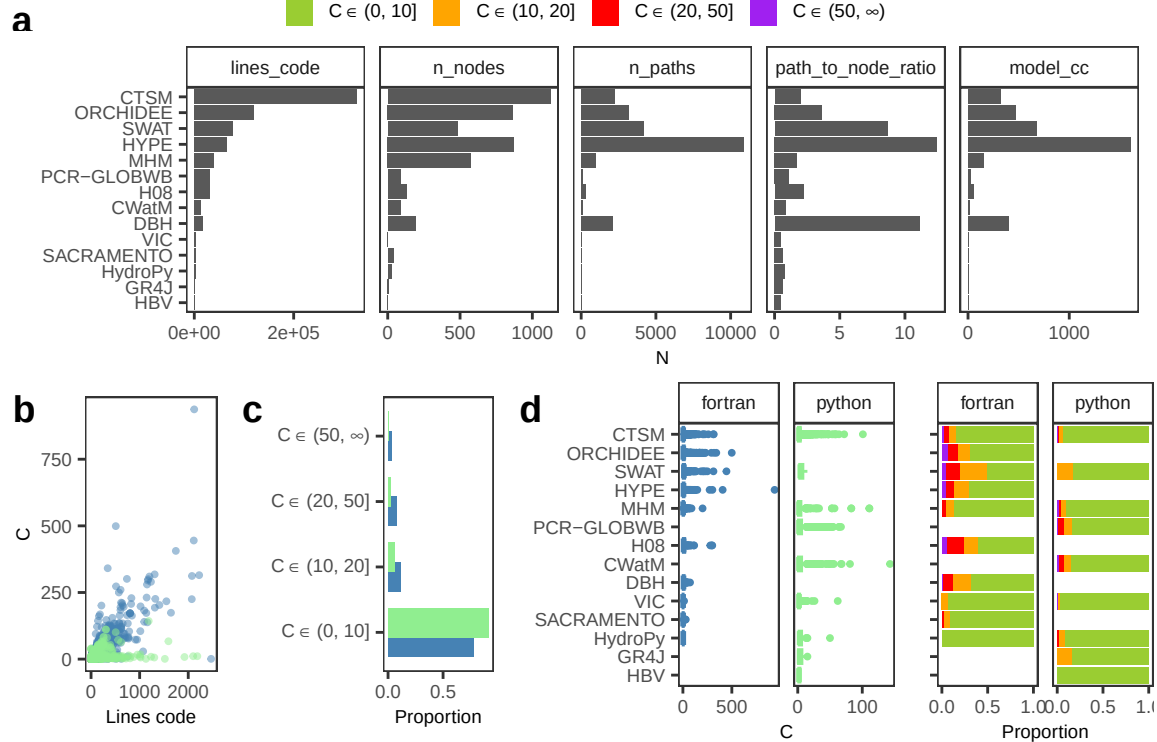
```
top_plot <- plot_grid(legend2, plot_descriptive, rel_heights = c(0.1, 0.9), ncol = 1,
  labels = "a")
```

```
## Warning: No shared levels found between `names(values)` of the manual scale and the
## data's fill values.
```

```
bottom <- plot_grid(plot_c_vs_loc, plot_bar_cyclomatic, plot_c_model,
  plot_bar_category, ncol = 4, rel_widths = c(0.2, 0.24, 0.34, 0.22),
  labels = c("b", "c", "d"))
```

```
## Warning: Removed 1195 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

```
plot_grid(top_plot, bottom, ncol = 1, rel_heights = c(0.52, 0.48), align = "h",
          axis = "tb")
```



PLOT FIGURES

Define language of models -----

```
python_models <- c("CWatM", "GR4J", "HBV", "PCR-GLOBWB")
fortran_models <- c("ORCHIDEE", "H08", "HYPE", "DBH", "SACRAMENTO")
python_and_fortran <- c("CTSM", "SWAT", "MHM", "HydroPy", "VIC")

all_graphs[, language := fcase(model %chin% python_models, "python",
                                model %chin% fortran_models, "fortran",
                                model %chin% python_and_fortran, "python+fortran",
                                default = NA_character_)]
```

Define specifications -----

```
plot_specs <- list(additive = list(paths_col = "paths_tbl_additive",
                                    out_col = "plot_obj_additive"),
                    power_mean = list(paths_col = "paths_tbl_power_mean",
                                       out_col = "plot_obj_power"))
```

Plot -----

```

# Thickness of edge: frequency across top-10 risky paths
# Color of edge: mean risk of paths using that edge

# Plot graphs -----

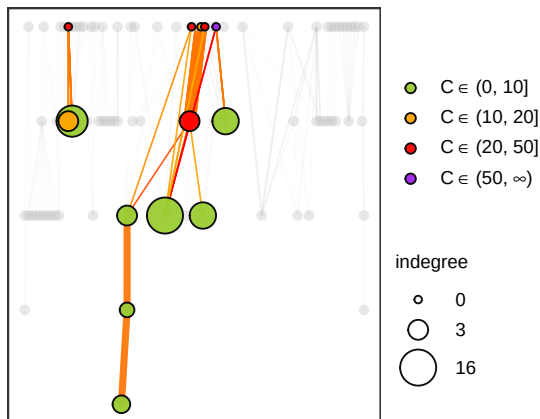
set.seed(seed)

for (nm in names(plot_specs)) {
  spec <- plot_specs[[nm]]

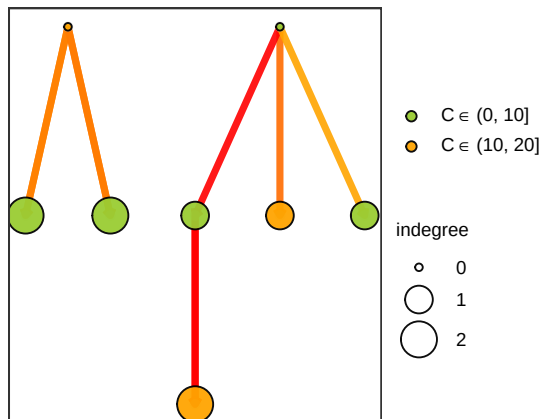
  all_graphs[, (spec$out_col) := mapply(FUN = plot_top_paths_fun, graph = graph,
    all_paths_out= get(spec$paths_col),
    model.name = model, language = language,
    SIMPLIFY = FALSE)]
}

```

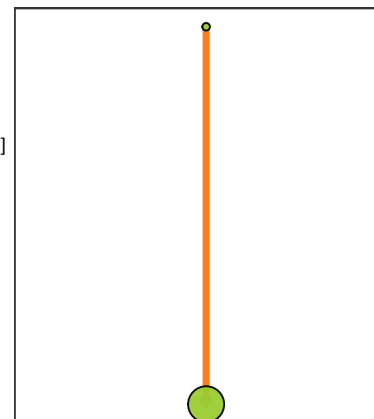
CWatM: python



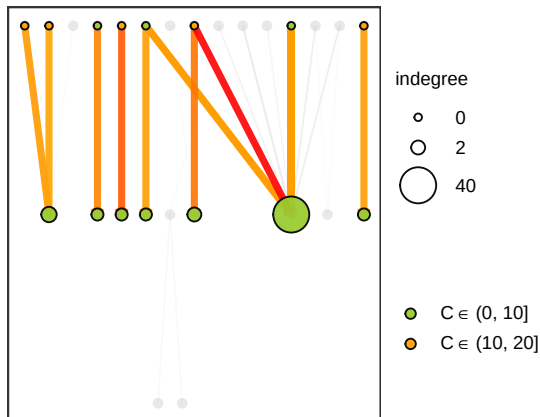
GR4J: python



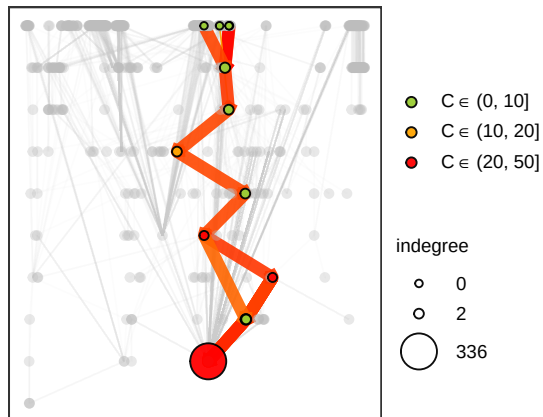
HBV: python



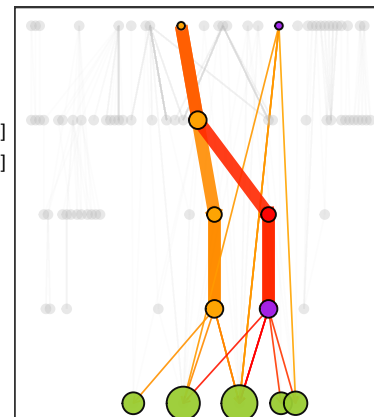
HydroPy: python+fortran



MHM: python+fortran

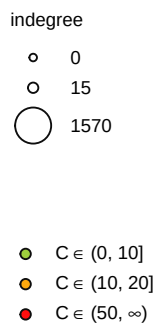
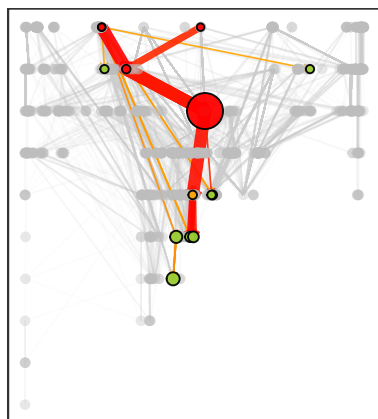


PCR-GLOBWB: python

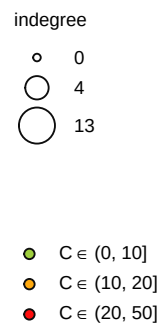
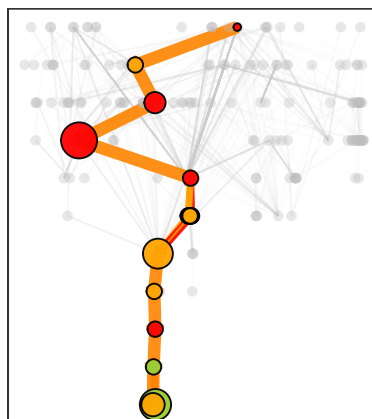


No paths in `paths\_tbl`; skipping plot for: VIC

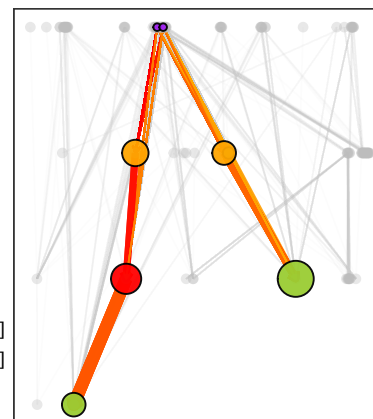
CTSM: python+fortran



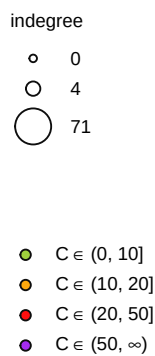
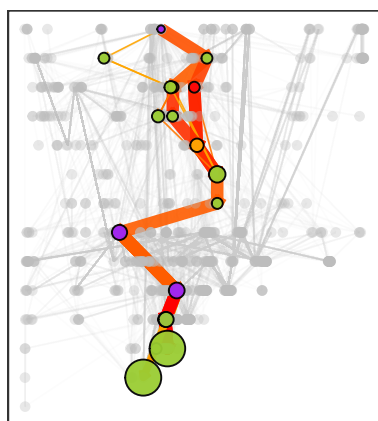
DBH: fortran



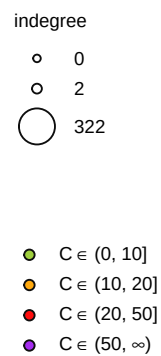
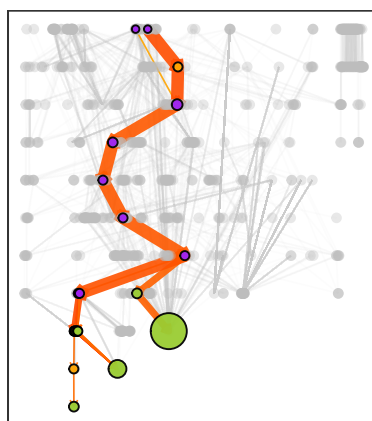
H08: fortran



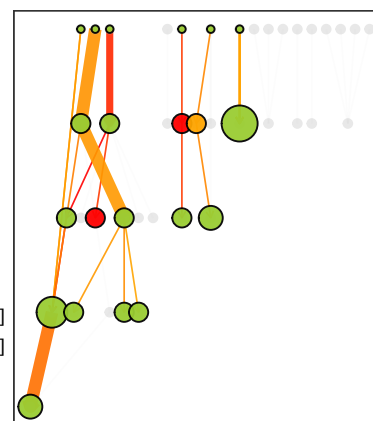
HYPE: fortran



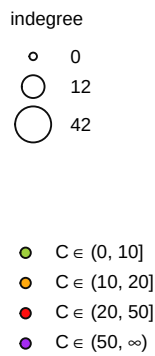
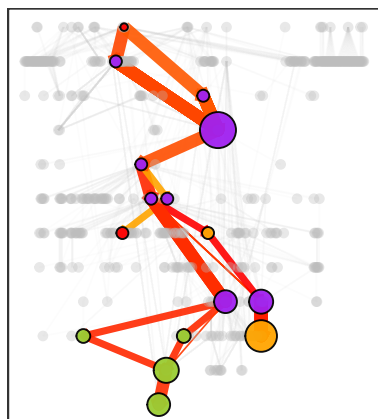
ORCHIDEE: fortran



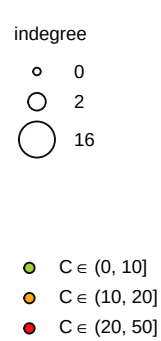
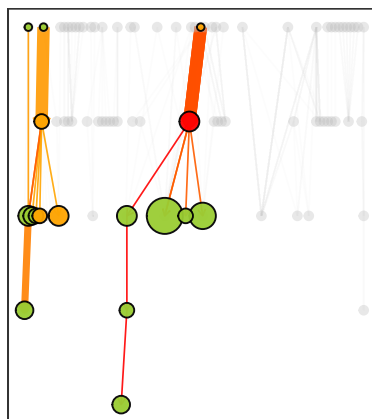
SACRAMENTO: fortran



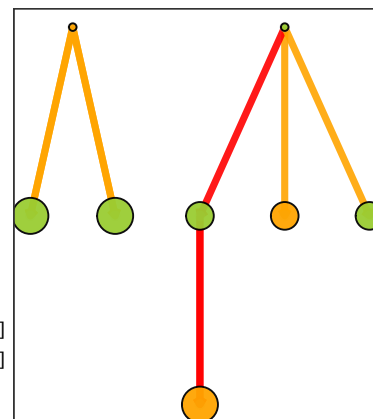
SWAT: python+fortran



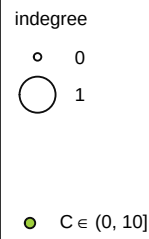
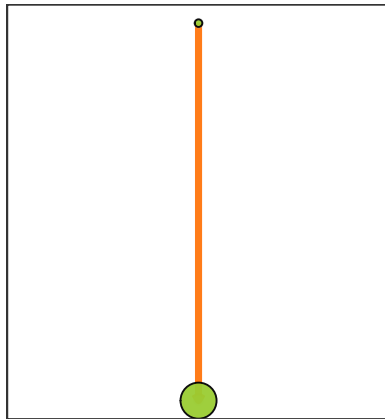
CWatM: python



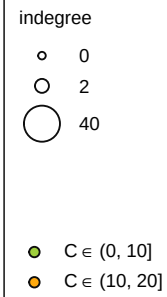
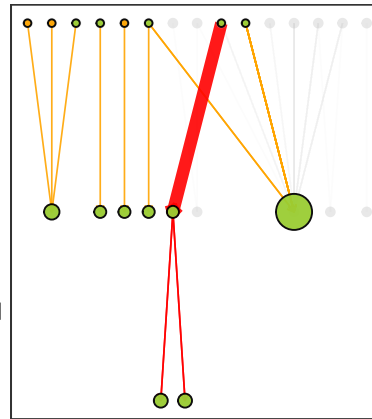
GR4J: python



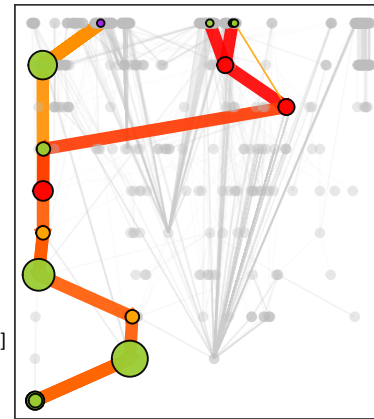
HBV: python



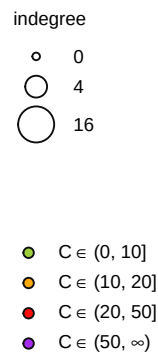
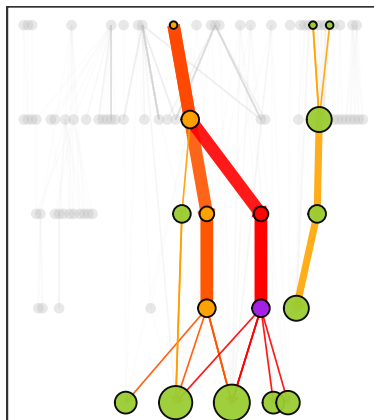
HydroPy: python+fortran



MHM: python+fortran

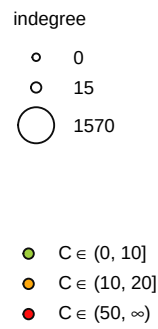
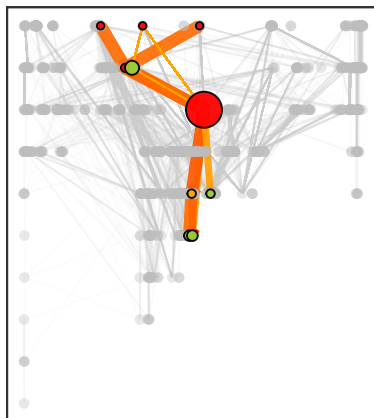


PCR-GLOBWB: python

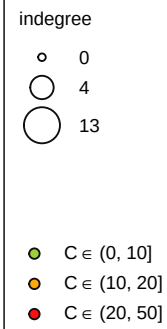
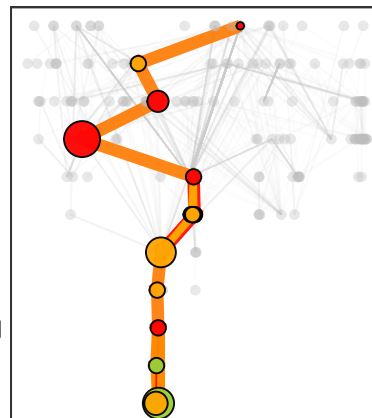


No paths in `paths\_tbl`; skipping plot for: VIC

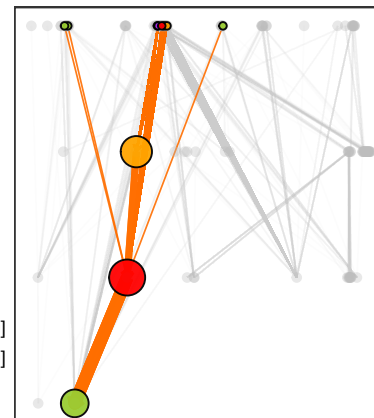
CTSM: python+fortran



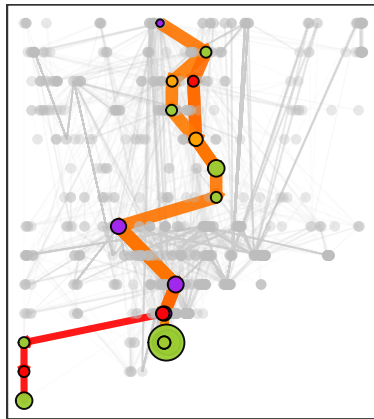
DBH: fortran



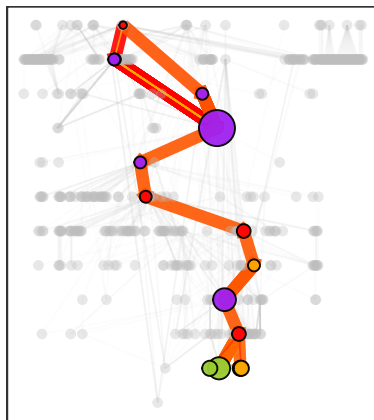
H08: fortran



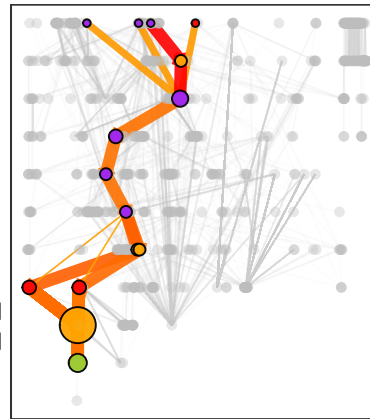
HYPE: fortran



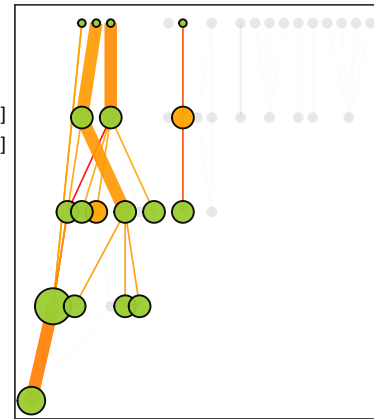
SWAT: python+fortran



ORCHIDEE: fortran



SACRAMENTO: fortran



Remove the fill legend -----

```
remove_fill_legend <- function(p) {
  p + guides(fill = "none")
}
```

```
plot_cols <- grep("^plot_obj_", names(all_graphs), value = TRUE)
```

```
for (col in plot_cols) {
  all_graphs[, (col) := lapply(get(col), remove_fill_legend)]}

```

Define models to plot -----

```
selected_models <- data.table(model = c("CTSM", "PCR-GLOBWB", "DBH", "HYPE",
    "ORCHIDEE", "SWAT", "CWatM", "MHM"))
```

Plot call graphs -----

```
tmp <- all_graphs[selected_models, on = .(model)]
plot_all_risky_paths <- plot_grid(plotlist = tmp$plot_obj_additive, ncol = 2, align = "hv")
plot_all_risky_paths <- plot_grid(legend2, plot_all_risky_paths, ncol = 1, rel_heights = c(0.1
```

```
# HOW MANY OF THE TOP N PATHS ARE THE SAME IN THE ADDITIVE AND POWER-MEAN? ###
```

```
top_n <- 10
top10 <- paths_us_all[, .SD[order(-P_median)][1:top_n], .(model, risk_form)]
```

```
# Separate the two risk definitions -----
```

```
top_add <- unique(top10[risk_form == "additive", .(model, path_id)])
top_pow <- unique(top10[risk_form == "power_mean", .(model, path_id)])
```

```
# Count overlaps per model -----
```

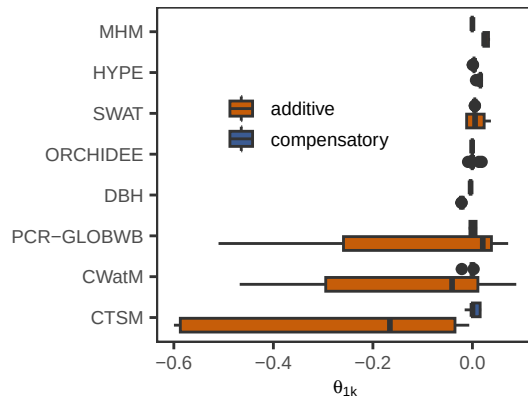
```
top_add[top_pow, on = .(model, path_id), .N, model] %>%
  na.omit() %>%
  .[selected_models, on = .(model)] %>%
  .[order(-N)]
```

```
##      model      N
##      <char> <int>
## 1:      CTSM     10
## 2: PCR-GLOBWB     10
## 3:       DBH     10
## 4:      HYPE     10
## 5: ORCHIDEE     10
## 6:       MHM     10
## 7:      SWAT      9
## 8:    CWatM      9
```

```
# PLOT DISTRIBUTION OF RISK SLOPE AND GINI FOR THE TOP TEN PATHS #####
```

```
plot_spread_risk <- paths_all[order(-path_risk_score), .SD[1:10], .(model, risk_form)] %>%
  .[selected_models, on = .(model)] %>%
  .[, risk_form:= fifelse(risk_form == "power_mean", "compensatory", risk_form)] %>%
  ggplot(., aes(reorder(model, risk_slope), risk_slope, fill = risk_form)) +
  scale_fill_manual(values = color_functional_forms, name = "") +
  geom_boxplot() +
  labs(x = "", y = expression(theta[1*k])) +
  coord_flip() +
  theme_AP() +
  theme(legend.position = c(0.4, 0.7))
```

```
plot_spread_risk
```

PATH-LEVEL RISK ACCOUNTED FOR THE TOP 5% NODES

To long format -----

```
paths_long <- paths_all[, .(node = unlist(path_nodes),
  path_risk_score = path_risk_score,
  gini_node_risk = gini_node_risk,
  risk_slope = risk_slope,
  risk_mean = risk_mean,
  risk_sum = risk_sum),
  .(model, path_id, risk_form)]
```

Aggregate at function level -----

```
node_from_paths <- paths_long[, .(n_paths = .N,
  mean_p_path = mean(path_risk_score, na.rm = TRUE),
  max_p_path = max(path_risk_score, na.rm = TRUE),
  sum_p_path = sum(path_risk_score, na.rm = TRUE),
  mean_gini = mean(gini_node_risk, na.rm = TRUE),
  mean_slope = mean(risk_slope, na.rm = TRUE),
  mean_risksum = mean(path_risk_score, na.rm = TRUE)),
  .(model, node, risk_form)]
```

Join with nodes -----

```
node_summary <- merge(node_from_paths, nodes_all[, .(name, model, risk_score, risk_form)],
  by.x = c("model", "node", "risk_form"),
  by.y = c("model", "name", "risk_form"), all.x = TRUE)
```

Calculate risk mass -----

```
node_summary[, risk_mass := mean_p_path * n_paths]
```

share of risk mass in top X% nodes, per model .-----

```
top_share <- function(X = 0.05) {
```

```

node_summary[!is.na(risk_mass) & risk_mass >= 0, {
  dt <- .SD[order(-risk_mass)]
  n_top <- max(1L, ceiling(.N * X))
  .(X = X, n_nodes = .N, n_top = n_top,
    share_risk_mass_topX = sum(dt$risk_mass[1:n_top]) / sum(dt$risk_mass))
},
.(model, risk_form)
]
}

```

Run function -----

```
tmp <- top_share(0.05)
```

```

table_top5 <- tmp %>%
  dcast(., model ~ risk_form, value.var = "share_risk_mass_topX") %>%
  .[order(-additive)]

```

```
table_top5
```

```

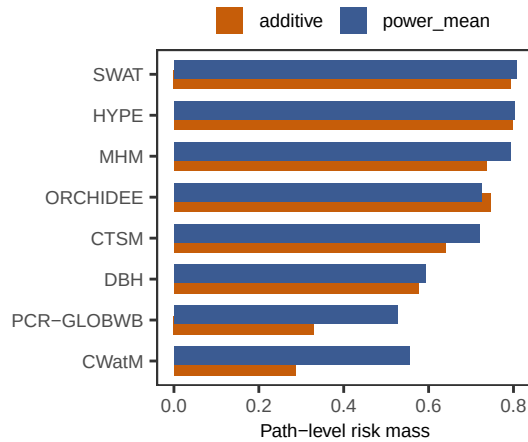
##      model  additive power_mean
##      <char>      <num>      <num>
##  1:      HYPE 0.7985366  0.8016064
##  2:      SWAT 0.7942209  0.8065609
##  3: ORCHIDEE 0.7460796  0.7251192
##  4:      MHM 0.7355383  0.7931651
##  5:      CTSM 0.6388660  0.7203974
##  6:      DBH 0.5763146  0.5924118
##  7:      HBV 0.5000000  0.5000000
##  8:      H08 0.4726677  0.6384604
##  9: PCR-GLOBWB 0.3300054  0.5273031
## 10:      CWatM 0.2862374  0.5544199
## 11:      GR4J 0.2726486  0.3333333
## 12:   HydroPy 0.2706942  0.6666667
## 13: SACRAMENTO 0.2418015  0.3183764

```

```

table_top5 %>%
  .[selected_models, on = .(model)] %>%
  melt(., measure.vars = c("additive", "power_mean")) %>%
  ggplot(., aes(reorder(model, value), value, fill = variable)) +
  geom_bar(stat = "identity", position = position_dodge(0.5)) +
  coord_flip() +
  labs(y = "Path-level risk mass", x = "") +
  scale_fill_manual(values = color_functional_forms, name = "") +
  theme_AP() +
  theme(legend.position = "top")

```



Rerun name of the functions that dominate the top 5% of risk mass -----

```
top_share_functions <- function(X = 0.05) {

  node_summary[
    !is.na(risk_mass) & risk_mass >= 0,
    {
      dt <- .SD[order(-risk_mass)]
      n_top <- max(1L, ceiling(.N * X))

      .(
        X = X,
        n_nodes = .N,
        n_top = n_top,
        share_risk_mass_topX = sum(dt$risk_mass[1:n_top]) / sum(dt$risk_mass),
        top_nodes = list(dt$node[1:n_top])
      )
    },
    by = .(model, risk_form)
  ]
}

tmp <- top_share_functions(0.05)
tmp
```

##	model	risk_form	X	n_nodes	n_top	share_risk_mass_topX
##	<char>	<char>	<num>	<int>	<num>	<num>
## 1:	CTSM	additive	0.05	1124	57	0.6388660
## 2:	CTSM	power_mean	0.05	1124	57	0.7203974
## 3:	CWatM	additive	0.05	84	5	0.2862374
## 4:	CWatM	power_mean	0.05	84	5	0.5544199
## 5:	DBH	additive	0.05	191	10	0.5763146
## 6:	DBH	power_mean	0.05	191	10	0.5924118
## 7:	GR4J	additive	0.05	8	1	0.2726486
## 8:	GR4J	power_mean	0.05	8	1	0.3333333

```

## 9:      H08      additive 0.05      129      7      0.4726677
## 10:     H08 power_mean 0.05      129      7      0.6384604
## 11:     HBV      additive 0.05        2      1      0.5000000
## 12:     HBV power_mean 0.05        2      1      0.5000000
## 13:     HYPE      additive 0.05     872     44      0.7985366
## 14:     HYPE power_mean 0.05     872     44      0.8016064
## 15:    HydroPy      additive 0.05      26      2      0.2706942
## 16:    HydroPy power_mean 0.05      26      2      0.6666667
## 17:      MHM      additive 0.05     565     29      0.7355383
## 18:      MHM power_mean 0.05     565     29      0.7931651
## 19:    ORCHIDEE      additive 0.05     865     44      0.7460796
## 20:    ORCHIDEE power_mean 0.05     865     44      0.7251192
## 21: PCR-GLOBWB      additive 0.05      89      5      0.3300054
## 22: PCR-GLOBWB power_mean 0.05      89      5      0.5273031
## 23: SACRAMENTO      additive 0.05      41      3      0.2418015
## 24: SACRAMENTO power_mean 0.05      41      3      0.3183764
## 25:      SWAT      additive 0.05     481     25      0.7942209
## 26:      SWAT power_mean 0.05     481     25      0.8065609
##      model risk_form      X n_nodes n_top share_risk_mass_topX
##      <char>      <char> <num>      <int> <num>      <num>
##
##
## 1:      clm_drv,initialize2,ModelAdvance,lnd_run,InitializeReal
## 2:      initialize2,clm_drv,hist_addfld1d,get_proc_bounds,allhistfldlist_addfld,Initi
## 3:      cbinding,loadsetclone,run_from_command_l
## 4:      loadsetclone,run_from_command_line,main,l
## 5:      Get_Grid_ATM,Read_Interpolate_data,LOTPS,(main:ETPTREND.f)
## 6:      Get_Grid_ATM,Read_Interpolate_data,LOTPS,LOCAL,(main:ETPTR
## 7:
## 8:
## 9:      read_binary,main_human,main_ftcs,wrt_e_binary,read_result
## 10:     wrte_binary,wrt_e_bints2,read_binary,read_result,main_hu
## 11:
## 12:
## 13:     HYSS,model,function_to_minim,run_model_crit,lineSearch_methods_calibration,run
## 14:     HYSS,model,function_to_minim,run_model_crit,lineSearch_methods_calibration,run
## 15:      log_cells.add_v
## 16:      routing_casca
## 17:      message,mhm_interface_init,mhm_eval,mhm_interface_run_prepare,r
## 18:      mhm_interface_init,caldat,dec2date,get_time_vector_and_select,read_n
## 19:      sechiba_main,slowproc_main,stomate_main,StomateLpj,ipsle
## 20:      sechiba_main,slowproc_main,stomate_main,StomateLpj,sechiba_initiali
## 21:      readPCRmapClone,singleTryReadPCRmapClone,getMapAttributesALL,main,r
## 22: readPCRmapClone,singleTryReadPCRmapClone,netcdf2PCRobjClone,singleTryNetcdf2PCRobjClone
## 23:      initialize_from_file,sac_initia
## 24:      initialize_from_file,sac_initi
## 25:      time_control,calsoft_control,command,calsoft_hyd_bfr,actions
## 26:      time_control,calsoft_control,command,actions,calsoft_hyd_bfr

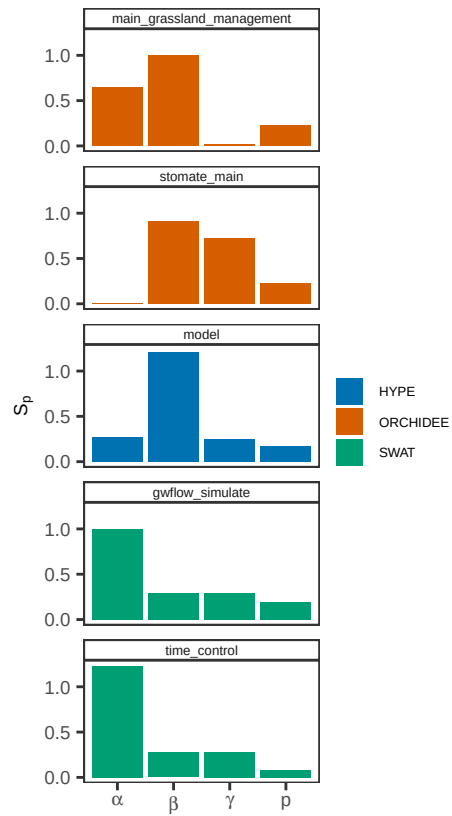
```

```
##
##
# SENSITIVITY OF SELECTED FUNCTIONS #####

functions_to_check <- c("main_grassland_management", "stomate_main", "model",
                        "gwflow_simulate", "time_control")

plot_sa_functions <- indices_all[name %in% functions_to_check] %>%
  .[, name:= factor(name, levels = functions_to_check)] %>%
  .[sensitivity == "Si"] %>%
  ggplot(., aes(parameters, original, fill = model)) +
  geom_bar(stat = "identity") +
  labs(x = "", y = expression(S[p])) +
  scale_fill_manual(values = c("#0072B2", "#D55E00", "#009E73"), name = "") +
  scale_x_discrete(labels = c(a_raw = expression(alpha),
                              b_raw = expression(beta),
                              c_raw = expression(gamma),
                              p = expression(p))) +
  facet_wrap(~name, ncol = 1) +
  theme_AP() +
  scale_y_continuous(breaks = breaks_pretty(n = 2)) +
  theme(legend.position = "right",
        legend.text = element_text(size = 5),
        strip.text.y.right = element_text(size = 5),
        strip.text.x.top = element_text(size = 4.5),
        strip.text.y = element_text(size = 7.4),
        strip.text = element_text(margin = margin(t = 1.5, b = 1.5)))

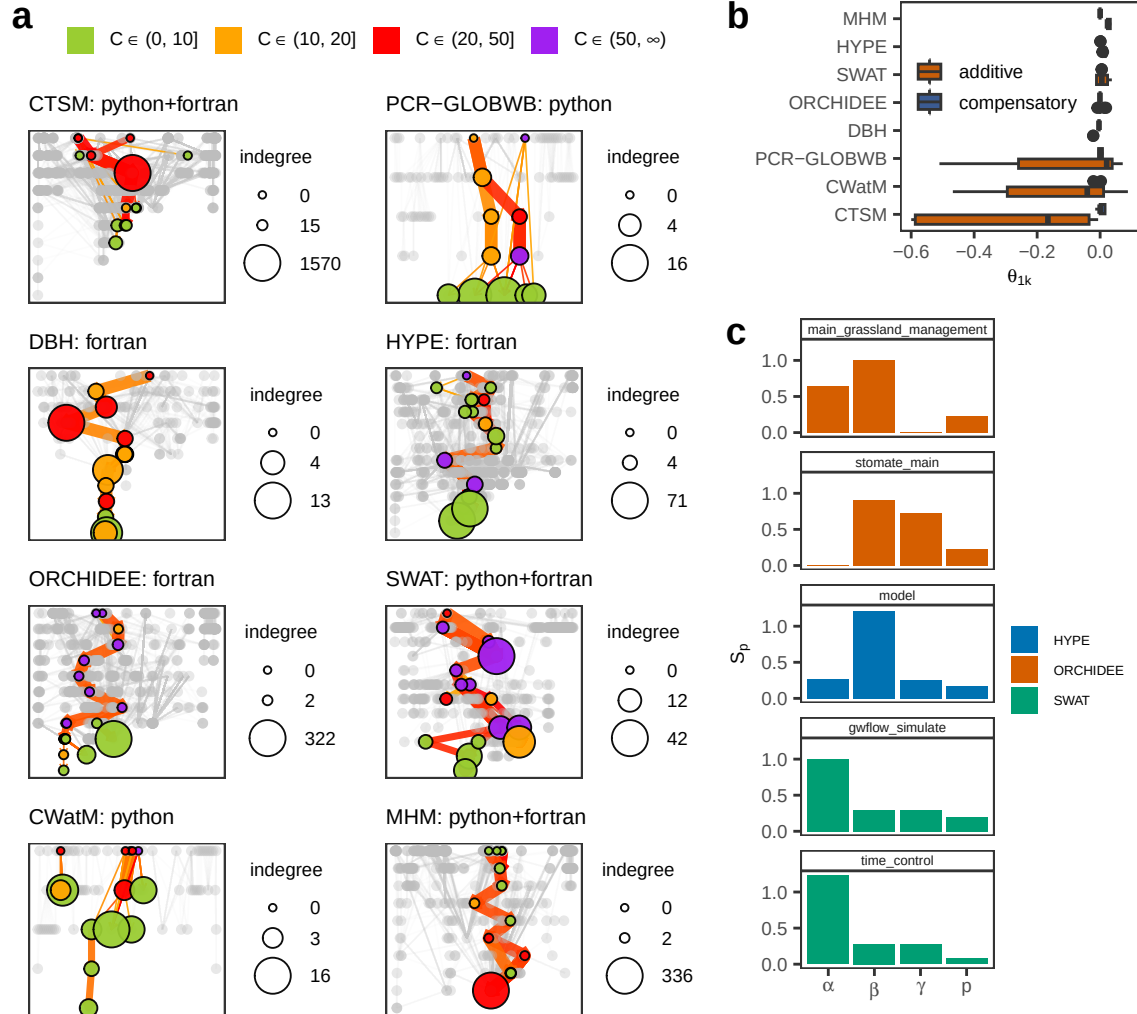
plot_sa_functions
```



```
# MERGE FIGURES #####

top <- plot_grid(plot_spread_risk, plot_sa_functions, ncol = 1,
                  rel_heights = c(0.3, 0.7), labels = c("b", "c"))

plot_grid(plot_all_risky_paths, top, rel_widths = c(0.62, 0.38),
           labels = c("a", ""))
```



WHICH PATHS IMPROVE IF A NODE'S RISK IS FIXED TO 0?

```
number_of_interest <- 30
```

```
all_graphs[!model == "VIC",
  path_fix_heatmap:= lapply(paths_tbl_additive,
    function(x) {
      path_fix_heatmap(all_paths_out = x,
        n_nodes = number_of_interest,
        k_paths = number_of_interest)
    }
  )]
```

Add name of the model by reference -----

```
all_graphs[, path_fix_heatmap:= Map(
  function(res, m) {if (is.null(res) || is.null(res$plot)) return(res)
    res$plot <- res$plot + ggtitle(m)
    res
```

```

    },
    path_fix_heatmap,
    model
  )]

# Plot -----

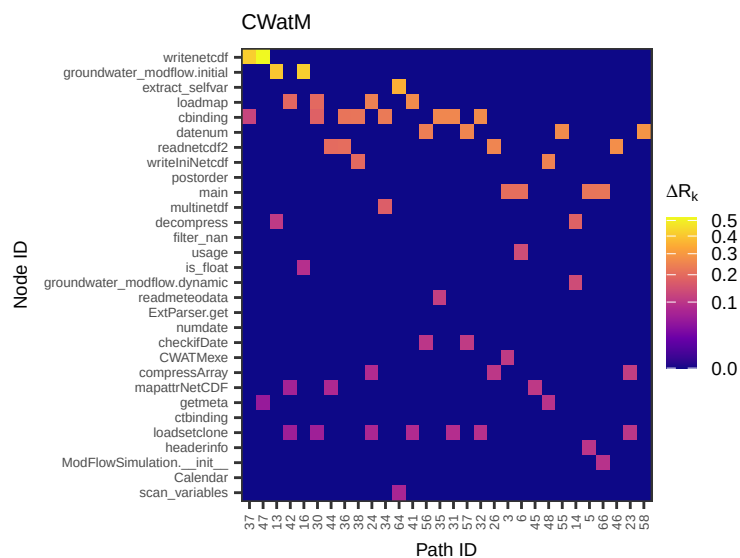
all_graphs$path_fix_heatmap

```

```

## [[1]]
## [[1]]$delta_tbl
## # A tibble: 900 x 3
##   node      path_id deltaR
##   <fct>    <fct>    <dbl>
## 1 writenetcdf 37      0.416
## 2 writenetcdf 47      0.521
## 3 writenetcdf 13       0
## 4 writenetcdf 42       0
## 5 writenetcdf 16       0
## 6 writenetcdf 30       0
## 7 writenetcdf 44       0
## 8 writenetcdf 36       0
## 9 writenetcdf 38       0
## 10 writenetcdf 24       0
## # i 890 more rows
##
## [[1]]$plot

```



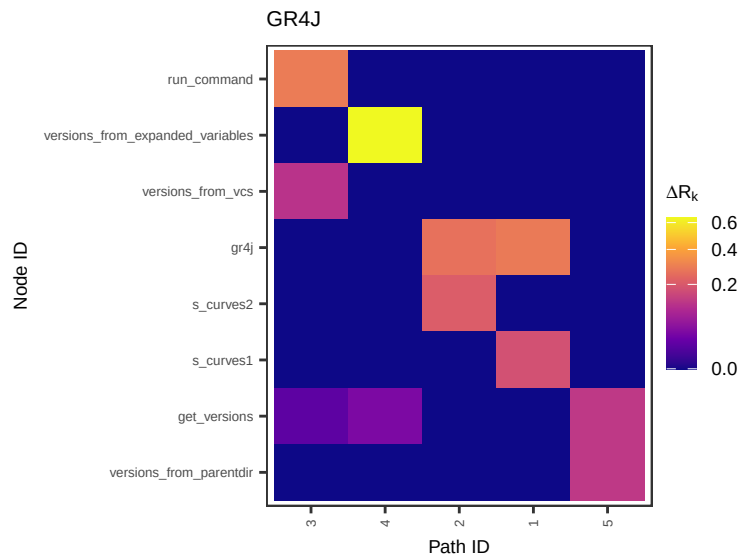
```

##
##
## [[2]]
## [[2]]$delta_tbl

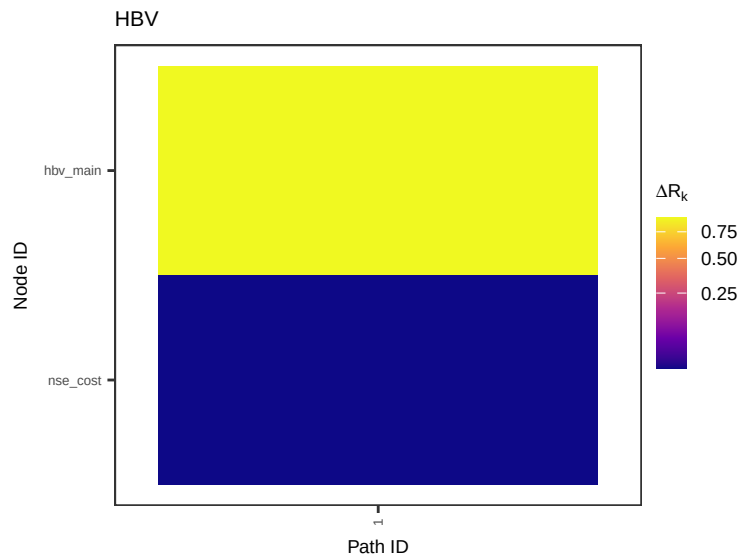
```



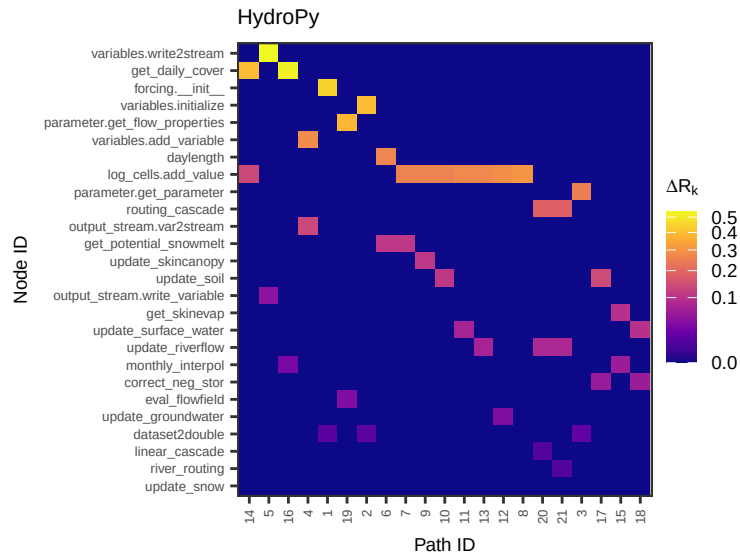
```
## # A tibble: 40 x 3
##   node                path_id deltaR
##   <fct>              <fct>    <dbl>
## 1 run_command        3      0.287
## 2 run_command        4      0
## 3 run_command        2      0
## 4 run_command        1      0
## 5 run_command        5      0
## 6 versions_from_expanded_variables 3      0
## 7 versions_from_expanded_variables 4     0.638
## 8 versions_from_expanded_variables 2      0
## 9 versions_from_expanded_variables 1      0
## 10 versions_from_expanded_variables 5      0
## # i 30 more rows
##
## [[2]]$plot
```



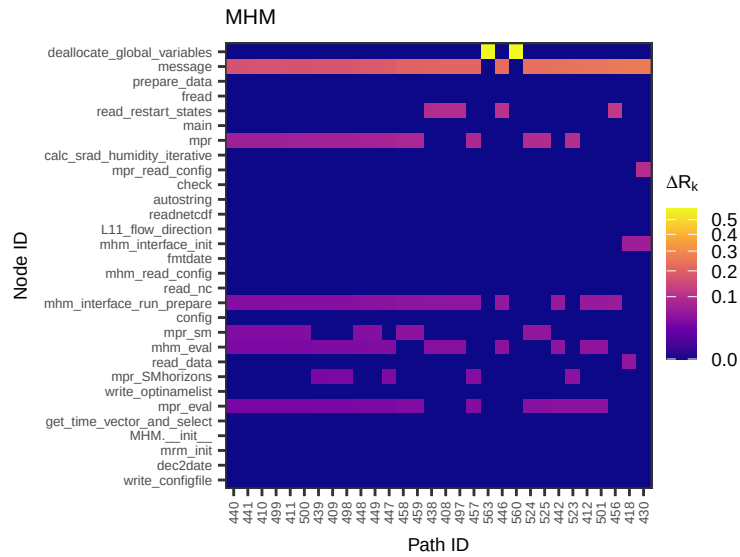
```
##
##
## [[3]]
## [[3]]$delta_tbl
## # A tibble: 2 x 3
##   node    path_id deltaR
##   <fct>   <fct>    <dbl>
## 1 hbv_main 1      0.902
## 2 nse_cost 1      0.00250
##
## [[3]]$plot
```



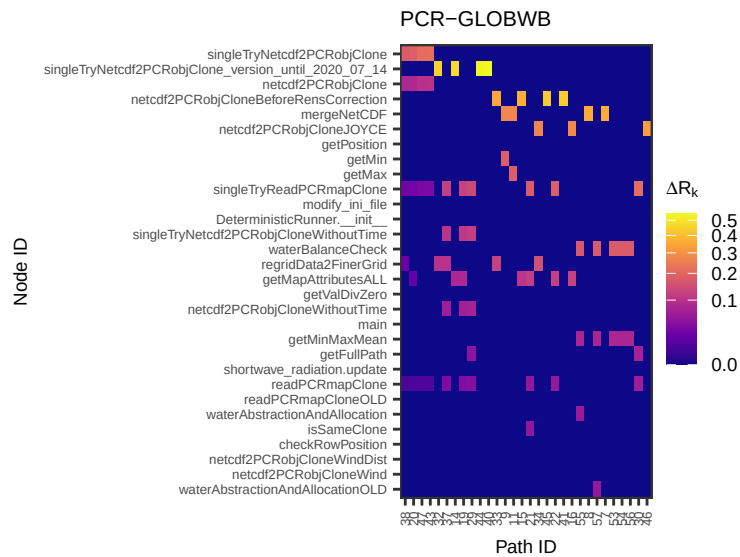
```
##
##
## [[4]]
## [[4]]$delta_tbl
## # A tibble: 546 x 3
##   node          path_id deltaR
##   <fct>         <fct>    <dbl>
## 1 variables.write2stream 14      0
## 2 variables.write2stream 5      0.536
## 3 variables.write2stream 16      0
## 4 variables.write2stream 4       0
## 5 variables.write2stream 1       0
## 6 variables.write2stream 19      0
## 7 variables.write2stream 2       0
## 8 variables.write2stream 6       0
## 9 variables.write2stream 7       0
## 10 variables.write2stream 9      0
## # i 536 more rows
##
## [[4]]$plot
```



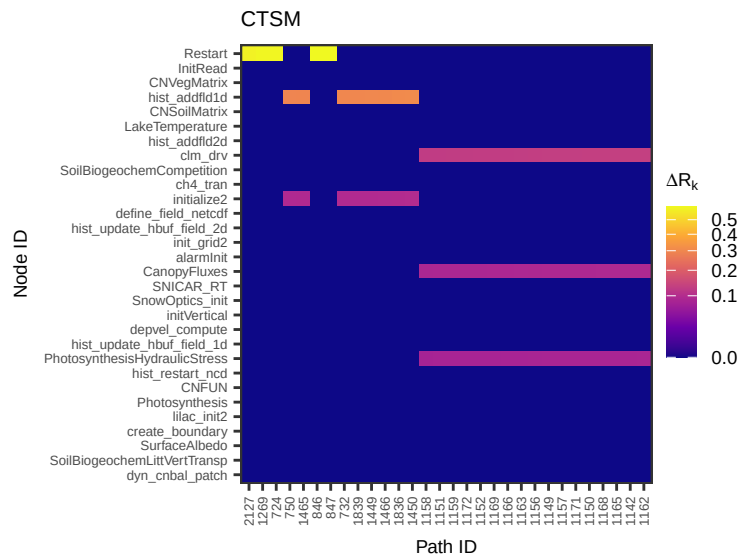
```
##
##
## [[5]]
## [[5]]$delta_tbl
## # A tibble: 900 x 3
##   node                path_id deltaR
##   <fct>              <fct>    <dbl>
## 1 deallocate_global_variables 440      0
## 2 deallocate_global_variables 441      0
## 3 deallocate_global_variables 410      0
## 4 deallocate_global_variables 499      0
## 5 deallocate_global_variables 411      0
## 6 deallocate_global_variables 500      0
## 7 deallocate_global_variables 439      0
## 8 deallocate_global_variables 409      0
## 9 deallocate_global_variables 498      0
## 10 deallocate_global_variables 448      0
## # i 890 more rows
##
## [[5]]$plot
```



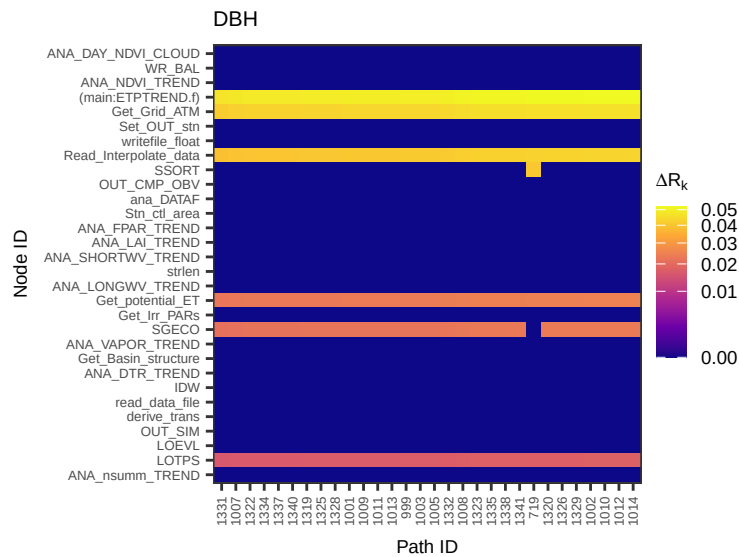
```
##
##
## [[6]]
## [[6]]$delta_tbl
## # A tibble: 900 x 3
##   node                                path_id deltaR
##   <fct>                                <fct>     <dbl>
## 1 singleTryNetcdf2PCRObjClone 38      0.175
## 2 singleTryNetcdf2PCRObjClone 20      0.184
## 3 singleTryNetcdf2PCRObjClone 47      0.213
## 4 singleTryNetcdf2PCRObjClone 43      0.216
## 5 singleTryNetcdf2PCRObjClone 32      0
## 6 singleTryNetcdf2PCRObjClone 37      0
## 7 singleTryNetcdf2PCRObjClone 14      0
## 8 singleTryNetcdf2PCRObjClone 19      0
## 9 singleTryNetcdf2PCRObjClone 29      0
## 10 singleTryNetcdf2PCRObjClone 44      0
## # i 890 more rows
##
## [[6]]$plot
```



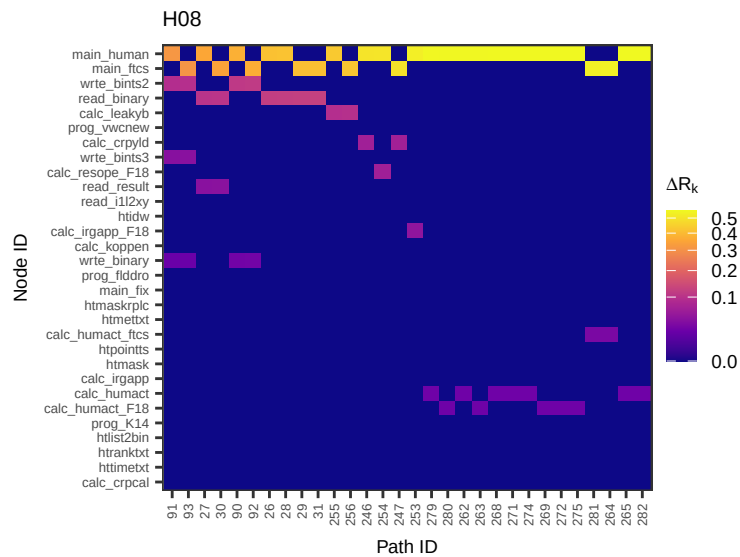
```
##
##
## [[7]]
## NULL
##
## [[8]]
## [[8]]$delta_tbl
## # A tibble: 900 x 3
##   node   path_id deltaR
##   <fct> <fct>   <dbl>
## 1 Restart 2127    0.579
## 2 Restart 1269    0.591
## 3 Restart 724     0.592
## 4 Restart 750     0
## 5 Restart 1465    0
## 6 Restart 846     0.600
## 7 Restart 847     0.600
## 8 Restart 732     0
## 9 Restart 1839    0
## 10 Restart 1449    0
## # i 890 more rows
##
## [[8]]$plot
```



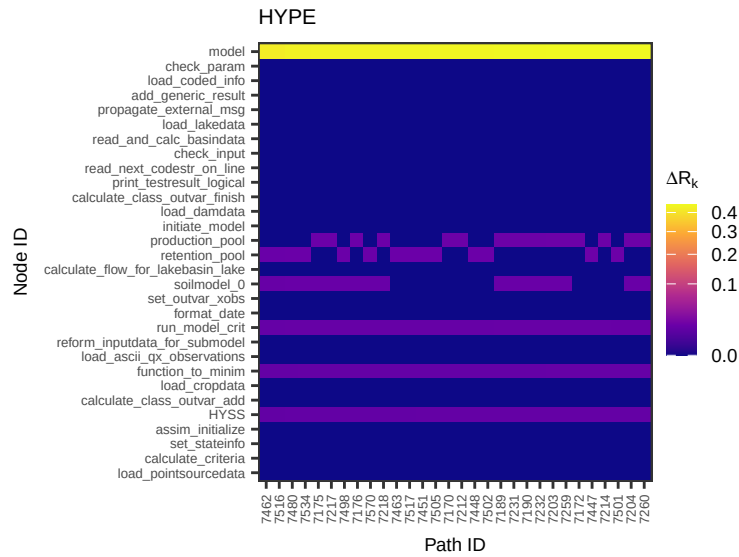
```
##
##
## [[9]]
## [[9]]$delta_tbl
## # A tibble: 900 x 3
##   node                path_id deltaR
##   <fct>                <fct>    <dbl>
## 1 ANA_DAY_NDVI_CLOUD 1331      0
## 2 ANA_DAY_NDVI_CLOUD 1007      0
## 3 ANA_DAY_NDVI_CLOUD 1322      0
## 4 ANA_DAY_NDVI_CLOUD 1334      0
## 5 ANA_DAY_NDVI_CLOUD 1337      0
## 6 ANA_DAY_NDVI_CLOUD 1340      0
## 7 ANA_DAY_NDVI_CLOUD 1319      0
## 8 ANA_DAY_NDVI_CLOUD 1325      0
## 9 ANA_DAY_NDVI_CLOUD 1328      0
## 10 ANA_DAY_NDVI_CLOUD 1001      0
## # i 890 more rows
##
## [[9]]$plot
```



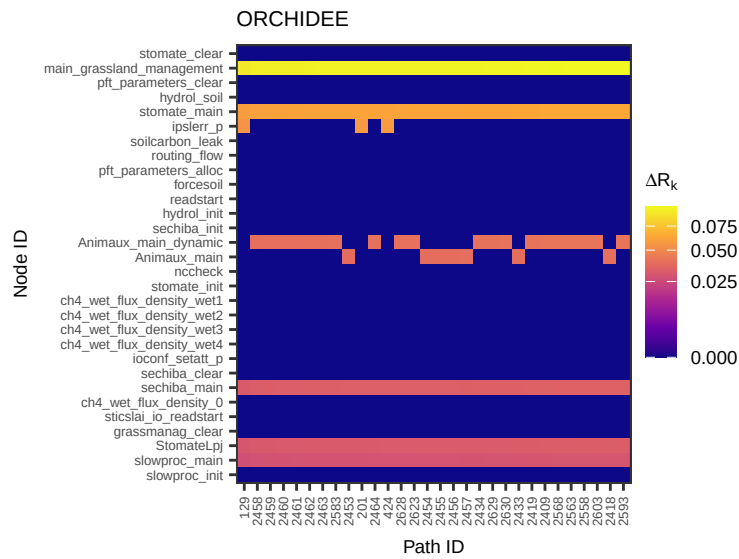
```
##
##
## [[10]]
## [[10]]$delta_tbl
## # A tibble: 900 x 3
##   node      path_id deltaR
##   <fct>    <fct>    <dbl>
## 1 main_human 91      0.314
## 2 main_human 93      0
## 3 main_human 27      0.354
## 4 main_human 30      0
## 5 main_human 90      0.376
## 6 main_human 92      0
## 7 main_human 26      0.413
## 8 main_human 28      0.419
## 9 main_human 29      0
## 10 main_human 31     0
## # i 890 more rows
##
## [[10]]$plot
```



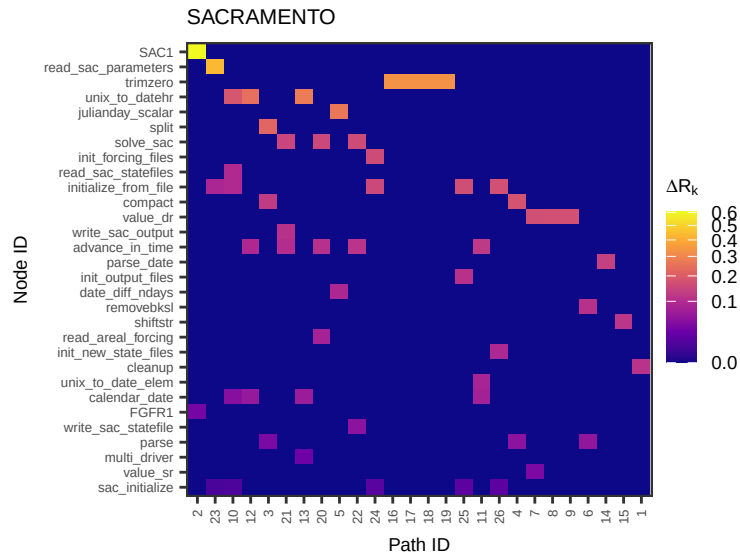
```
##
##
## [[11]]
## [[11]]$delta_tbl
## # A tibble: 900 x 3
##   node path_id deltaR
##   <fct> <fct>     <dbl>
## 1 model 7462     0.419
## 2 model 7516     0.420
## 3 model 7480     0.430
## 4 model 7534     0.430
## 5 model 7175     0.431
## 6 model 7217     0.432
## 7 model 7498     0.432
## 8 model 7176     0.432
## 9 model 7570     0.432
## 10 model 7218     0.432
## # i 890 more rows
##
## [[11]]$plot
```

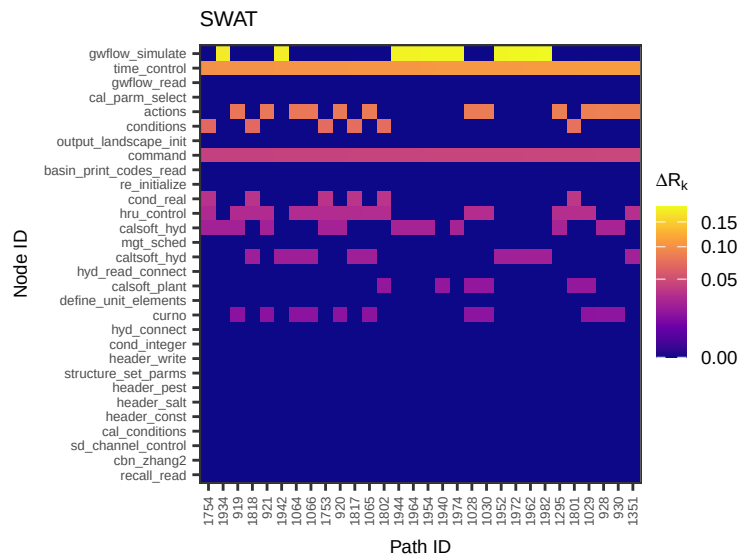
```
##
##
## [[12]]
## [[12]]$delta_tbl
## # A tibble: 900 x 3
##   node          path_id deltaR
##   <fct>         <fct>    <dbl>
## 1 stomate_clear 129      0
## 2 stomate_clear 2458     0
## 3 stomate_clear 2459     0
## 4 stomate_clear 2460     0
## 5 stomate_clear 2461     0
## 6 stomate_clear 2462     0
## 7 stomate_clear 2463     0
## 8 stomate_clear 2583     0
## 9 stomate_clear 2453     0
## 10 stomate_clear 201      0
## # i 890 more rows
##
## [[12]]$plot
```



```
##
##
## [[13]]
## [[13]]$delta_tbl
## # A tibble: 780 x 3
##   node path_id deltaR
##   <fct> <fct>     <dbl>
## 1 SAC1  2         0.603
## 2 SAC1 23         0
## 3 SAC1 10         0
## 4 SAC1 12         0
## 5 SAC1  3         0
## 6 SAC1 21         0
## 7 SAC1 13         0
## 8 SAC1 20         0
## 9 SAC1  5         0
## 10 SAC1 22         0
## # i 770 more rows
##
## [[13]]$plot
```



```
##
##
## [[14]]
## [[14]]$delta_tbl
## # A tibble: 900 x 3
##   node          path_id deltaR
##   <fct>         <fct>   <dbl>
## 1 gwflow_simulate 1754     0
## 2 gwflow_simulate 1934    0.173
## 3 gwflow_simulate 919      0
## 4 gwflow_simulate 1818     0
## 5 gwflow_simulate 921      0
## 6 gwflow_simulate 1942    0.176
## 7 gwflow_simulate 1064     0
## 8 gwflow_simulate 1066     0
## 9 gwflow_simulate 1753     0
## 10 gwflow_simulate 920      0
## # i 890 more rows
##
## [[14]]$plot
```



3 Analysis of git logs

METRICS AT THE FILE AND FUNCTION LEVEL

```
folder <- "./datasets/git_logs"
```

Get names of files -----

```
csv_files <- list.files(path = folder, pattern = "_2\\.csv$", full.names = TRUE)
csv_files
```

```
## [1] "./datasets/git_logs/CTSM_fortran_results-arnald_2.csv"
## [2] "./datasets/git_logs/CTSM_python_results-arnald_2.csv"
## [3] "./datasets/git_logs/CWatM_python_results-arnald_2.csv"
## [4] "./datasets/git_logs/PCR-GLOBWB_python_results-arnald_2.csv"
## [5] "./datasets/git_logs/VIC_fortran_results-arnald_2.csv"
## [6] "./datasets/git_logs/VIC_python_results-arnald_2.csv"
```

Merge and flatten -----

```
logs_dt <- lapply(csv_files, fread) %>%
  lapply(., function(x) x[, .(model, language, `function`, cyclomatic_complexity, number_changes,
    change_per_days, age_days)]) %>%
  rbindlist() %>%
  na.omit() %>%
  setnames(., "function", "node")
```

Create and run function -----

```
spearman_fun <- function(dt) {
```

```

cor.test(dt[, cyclomatic_complexity],
         dt[, number_changes], method = "spearman")$estimate[[1]]
}

rho_values <- logs_dt[, .(rho = spearman_fun(.SD)), model]

## Warning in cor.test.default(dt[, cyclomatic_complexity], dt[, number_changes],
## : Cannot compute exact p-value with ties
## Warning in cor.test.default(dt[, cyclomatic_complexity], dt[, number_changes],
## : Cannot compute exact p-value with ties
## Warning in cor.test.default(dt[, cyclomatic_complexity], dt[, number_changes],
## : Cannot compute exact p-value with ties
## Warning in cor.test.default(dt[, cyclomatic_complexity], dt[, number_changes],
## : Cannot compute exact p-value with ties
## Warning in cor.test.default(dt[, cyclomatic_complexity], dt[, number_changes],
## : Cannot compute exact p-value with ties

# Create dt to position labels -----

pos_df <- logs_dt %>%
  filter(is.finite(cyclomatic_complexity), is.finite(number_changes)) %>%
  group_by(model) %>%
  summarise(x = min(cyclomatic_complexity, na.rm = TRUE),
            y = max(number_changes, na.rm = TRUE), .groups = "drop")

# join rho and make label text -----

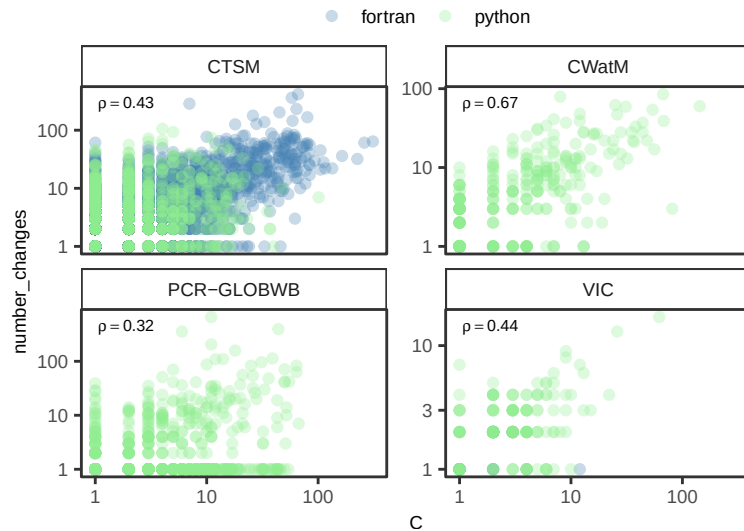
lab_df <- rho_values %>%
  left_join(pos_df, by = "model") %>%
  mutate(label = ifelse(is.na(rho), "rho==NA", paste0("rho==", round(rho, 2))))

# Plot -----

plot_logs <- logs_dt %>%
  ggplot(aes(cyclomatic_complexity, number_changes, color = language)) +
  geom_point(alpha = 0.3) +
  scale_color_manual(values = color_languages, name = "") +
  scale_x_log10() +
  scale_y_log10() +
  labs(x = expression(C), y = "number_changes") +
  facet_wrap(~model, scales = "free_y") +
  geom_text(data = lab_df %>% na.omit(),
            aes(x = x, y = y, label = label), inherit.aes = FALSE,
            hjust = -0.05, vjust = 1.2, size = 2, parse = TRUE) +
  coord_cartesian(clip = "off") +
  theme_AP() +
  theme(legend.position = "top")

plot_logs

```



ARE REFACTORING ACTIONS MORE COMMON IN RISKY PATHS?

```
models_to_check <- unique(logs_dt$model)
```

Extract name of functions in top ten risky paths per model -----

```
list_nodes <- paths_all %>%
  data.table() %>%
  .[order(-path_risk_score), .SD[1:10], model] %>%
  .[model %in% models_to_check] %>%
  na.omit() %>%
  .[, .(node = unlist(path_nodes)), model]
```

keep only unique risk nodes

```
risk_nodes <- unique(list_nodes[, .(model, node)])
risk_nodes[, high_risk_path := TRUE]
```

Filter out and keep only models that have log data -----

```
logs_dt <- logs_dt[model %in% models_to_check]
```

left join into logs_dt -----

```
logs_dt[risk_nodes, high_risk_path := i.high_risk_path,
  on = .(model, node)]
```

Turn NAs into FALSE -----

```
logs_dt[is.na(high_risk_path), high_risk_path := FALSE]
```

*# We now quantify effect size using the common-language statistic A,
defined as the probability that a randomly selected high-risk function*

has higher churn than a randomly selected non-risk function. -----

```
wilcox_dt <- logs_dt[is.finite(number_changes),
  {
    x <- number_changes[high_risk_path]
    y <- number_changes[!high_risk_path]

    n_x <- length(x)
    n_y <- length(y)

    if (n_x >= 3L && n_y >= 3L) {

      wt <- wilcox.test(x, y, alternative = "greater", exact = FALSE)

      # Wilcoxon rank-sum statistic (sum of ranks for x)-----
      W <- as.numeric(wt$statistic)

      # Convert to Mann-Whitney U -----
      U <- W - n_x * (n_x + 1) / 2

      # Common-language effect size (A statistic) -----
      A <- U / (n_x * n_y)

      .(
        p_value = wt$p.value,
        A_common_language = A,
        n_risk = n_x,
        n_nonrisk = n_y,
        median_risk = median(x),
        median_nonrisk = median(y)
      )
    } else {

      .(
        p_value = NA_real_,
        A_common_language = NA_real_,
        n_risk = n_x,
        n_nonrisk = n_y,
        median_risk = if (n_x) median(x) else NA_real_,
        median_nonrisk = if (n_y) median(y) else NA_real_
      )
    }
  },
model
```

]

```
wilcox_dt
```

```
##          model          p_value A_common_language n_risk n_nonrisk median_risk
##          <char>          <num>          <num>    <int>    <int>    <num>
## 1:         CTSM 2.649807e-06          0.6783021        50      3363        10.0
## 2:         CWatM 1.097483e-03          0.6967666        16       317        10.0
## 3: PCR-GLOBWB 1.332338e-02          0.5935438        24       859         1.5
## 4:         VIC          NA          NA          0       203          NA
## median_nonrisk
##          <num>
## 1:          4
## 2:          3
## 3:          1
## 4:          2
```

```
# FUNCTIONS TO SELECT THE TOP TEN RISKY PATHS PER MODEL AND
# PRINT THEM OUT FOR LATEX #####
```

```
# Top ten per model -----
```

```
tmp <- paths_all[risk_function == "additive"] %>%
  .[order(-path_risk_score), .SD[1:10], model] %>%
  .[, .(model, path_str)] %>%
  na.omit() %>%
  split(., .$model)
```

```
to_tex_list_fun(tmp)
```

```
# Random sample from bottom 20% per model -----
```

```
set.seed(seed)
```

```
tmp2 <- paths_additive[, {n_bot <- ceiling(.N * 0.2)}
.SD[order(path_risk_score)] %>%
  .[sample(seq_len(n_bot), size = min(5, n_bot))]
}, model] %>%
  .[, .(model, path_str)] %>%
  na.omit() %>%
  split(., .$model)
```

```
to_tex_list_fun(tmp2)
```


4 Session information

```
# SESSION INFORMATION #####
```

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.2
##
## Matrix products: default
## BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] tools parallel stats graphics grDevices utils datasets
## [8] methods base
##
## other attached packages:
## [1] softwareRisk_0.1.0 benchmarkme_1.0.8 sensobol_1.1.6 ggraph_2.2.2
## [5] foreach_1.5.2 igraph_2.2.1 tidygraph_1.3.1 here_1.0.2
## [9] tidytext_0.4.3 ggrepel_0.9.6 readxl_1.4.5 cowplot_1.2.0.9000
## [13] scales_1.4.0 openxlsx_4.2.8 lubridate_1.9.4 forcats_1.0.1
## [17] stringr_1.6.0 dplyr_1.1.4 purrr_1.2.1 readr_2.1.5
## [21] tidyr_1.3.2 tibble_3.3.1 ggplot2_4.0.1.9000 tidyverse_2.0.0
## [25] data.table_1.18.0
##
## loaded via a namespace (and not attached):
## [1] tidyselect_1.2.1 viridisLite_0.4.2 farver_2.1.2
## [4] viridis_0.6.5 S7_0.2.1 fastmap_1.2.0
## [7] tweenr_2.0.3 janeaustenr_1.0.0 digest_0.6.39
## [10] timechange_0.3.0 lifecycle_1.0.5 tokenizers_0.3.0
## [13] magrittr_2.0.4 compiler_4.5.2 rlang_1.1.7
## [16] utf8_1.2.6 yaml_2.3.12 knitr_1.51
## [19] labeling_0.4.3 graphlayouts_1.2.2 RColorBrewer_1.1-3
## [22] withr_3.0.2 grid_4.5.2 polyclip_1.10-7
## [25] iterators_1.0.14 MASS_7.3-65 tinytex_0.58
## [28] cli_3.6.5 rmarkdown_2.30 generics_0.1.4
## [31] rstudioapi_0.17.1 httr_1.4.7 tzdb_0.5.0
## [34] cachem_1.1.0 ggforce_0.5.0 cellranger_1.1.0
## [37] vctrs_0.6.5 Matrix_1.7-4 hms_1.1.3
## [40] ineq_0.2-13 glue_1.8.0 benchmarkmeData_1.0.4
```

```
## [43] codetools_0.2-20      stringi_1.8.7      gtable_0.3.6
## [46] pillar_1.11.1         htmltools_0.5.9    R6_2.6.1
## [49] Rdpack_2.6.4          doParallel_1.0.17  rprojroot_2.1.1
## [52] evaluate_1.0.5        lattice_0.22-7     rbibutils_2.4
## [55] SnowballC_0.7.1       memoise_2.0.1      Rcpp_1.1.1
## [58] zip_2.3.3             gridExtra_2.3      xfun_0.55
## [61] pkgconfig_2.0.3
```

```
## Return the machine CPU -----
```

```
cat("Machine:      "); print(get_cpu()$model_name)
```

```
## Machine:
```

```
## [1] "Apple M1 Max"
```

```
## Return number of true cores -----
```

```
cat("Num cores:    "); print(parallel::detectCores(logical = FALSE))
```

```
## Num cores:
```

```
## [1] 10
```

```
## Return number of threads -----
```

```
cat("Num threads: "); print(parallel::detectCores(logical = FALSE))
```

```
## Num threads:
```

```
## [1] 10
```